DevOps Question Bank
CI/CD, Jenkins, GitOps & Ansible

79 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
CI/CD, Jenkins, GitOps & Ansible	38	23	18	79

Questions in this topic

Basic **6**

Q1. What is the difference between CI, CD and GitOps? . . . 6

Q2. What triggers your pipeline, and how do you trigger a Jenkins job automatically? . . . 7

Q3. What are webhooks? . . . 8

Q4. What is a Jenkins agent (worker node)? Where does a pipeline run and where do you define agents? 9

Q5. What is the file structure of a Jenkinsfile, and how do you use it in your project? . . . 10

Q6. What is a multibranch pipeline? . . . 10

Q7. How do you define and invoke pipelines in Jenkins? . . . 11

Q8. What are the types of jobs in Jenkins? . . . 12

Q9. How do you configure GitHub in Jenkins for Freestyle and Pipeline jobs? . . . 13

Q10. How do you know the whole Jenkins pipeline actually executed successfully? . . . 14

Q11. What types of artifacts are generated in your pipeline? . . . 15

Q12. What is Blue-Green deployment vs Canary deployment? Explain with examples. . . . 16

Q13. How do you handle build failures? . . . 17

Q14. What branching strategy do you follow, and how do you handle a production bug? . . . 18

Q15. What is the difference between git merge and git rebase? When do you use each? . . . 19

Q16. What is git stash used for? . . . 20

Q17. How do you store credentials securely in Jenkins? . . . 21

Q18. Azure DevOps: How do you enforce branch policies and PR validation in Azure Repos? . . . 22

Q19. Azure DevOps: What is the purpose of YAML pipeline templates? . . . 23

Q20. Ansible: What is an Ansible playbook? . . . 24

Q21. Ansible: What is the difference between push and pull configuration management? . . . 25

Q22. Ansible: How do you manage Ansible inventory? . . . 26

Q23. Ansible: How do you work with modules and roles? . . . 26

Q24. Ansible: How do you use handlers in a playbook? . . . 27

Q25. Ansible: How do you store secrets securely in Ansible? . . . 28

Q26. Ansible: How do you run your Ansible projects? . . . 28

Q27. What is the difference between Scripted and Declarative Jenkins pipelines, and when would you use each? . . . 29

Q28. How do you set up Jenkins from scratch on a fresh server? . . . 30

Q29. How do you install Jenkins plugins, and which plugins are most common in a CI/CD setup? . . . 31

Q30. Git: what is the "detached HEAD" state, what causes it, and how do you fix it? . . . 32

Q31. Git/GitHub: how do you give a specific team member access to a repository, and grant them write permission? . . . 33

Q32. Git/GitHub: what is the difference between public and private repositories, and when would you use each? . . . 33
Q33. Git: how does squash work, when do you use it, and which commands do you run? . . . 34
Q34. Git: what is the difference between Git and GitHub? . . . 35
Q35. Git: what are the essential Git commands, and what does each do? . . . 36
Q36. Git: what is the difference between git push, git fetch and git pull? . . . 37
Q37. Git: how do you recover deleted changes (lost commits, a deleted branch, discarded files)? . . . 38
Q38. Git: which useful Git commands are often ignored? . . . 38

Advanced **39**

Q39. Explain your CI/CD pipeline architecture, including approvals. . . . 39
Q40. Explain how a typical AWS-native CI/CD pipeline works end to end. . . . 40
Q41. What stages do you define in Jenkins, and how do you ensure full quality checks? . . . 41
Q42. Write a CI/CD Jenkins pipeline that you have implemented in your project. . . . 42
Q43. What kinds of applications do you deploy through Jenkins, and with which deployment tools? . . . 44
Q44. How do you handle the CD aspect and promote builds across environments (Dev -> Staging -> Prod)? . . . 45
Q45. How do you connect Jenkins to AWS securely and push Docker images for multiple services to ECR? .
Q46. How do Jenkins and a Kubernetes cluster communicate? . . . 47
Q47. How can Jenkins be integrated with other DevOps tools? . . . 48
Q48. What are Jenkins Shared Libraries, and how do you use them to reduce or modularize a large Jenkinsfile? . . . 49
Q49. What rollback strategies do you use, and how do you version artifacts? . . . 50
Q50. What are secure pipeline (DevSecOps) practices? . . . 52
Q51. How do you secure secrets across CI/CD pipelines? . . . 52
Q52. How do you ensure secure and dynamic secret rotation in Azure DevOps pipelines? . . . 53
Q53. Azure DevOps: How do you secure access to artifacts and feeds (Azure Artifacts)? . . . 54
Q54. How do you manage deployments and rollbacks for 100+ microservices on EKS/ECS? . . . 55
Q55. How do you ensure zero-downtime deployments? Share a real example. . . . 56
Q56. How would you optimize a slow CI/CD pipeline (e.g. 40 minutes for a small change)? . . . 57
Q57. How would you implement blue/green deployment with rollback on Azure using Terraform and pipelines? . . . 59
Q58. Compare deployment strategies: in-place, rolling, immutable, traffic splitting (canary) and blue/green. 60
Q59. How do you implement environment-specific CI/CD pipelines for 10+ microservices, and how do you scale and manage them? . . . 62
Q60. How would you design a GitOps pipeline for a microservices application? Which tools, what flow, and how do you roll back? . . . 63
Q61. How do you handle databases in blue-green deployments? With critical data, how do you keep the database consistent and avoid issues during switchover? . . . 66

Scenario-Based **68**

Q62. The Jenkins controller goes down while builds are running on agents. What happens and what do you do? . . . 68

Q63. The Jenkins master (controller) is running out of disk space. What actions would you take? . . . 69

Q64. Build artifacts are not getting uploaded to the artifact repository, or Jenkins jobs fail randomly at the artifact upload step. How do you investigate? . . . 70

Q65. A Jenkins pipeline fails randomly after long runs. How do you find the root cause? . . . 71

Q66. The Jenkins pipeline runs, but the build does not actually happen. What could be wrong? . . . 72

Q67. Jenkins credentials were exposed accidentally. What is your immediate action plan? . . . 73

Q68. CI/CD logs show flaky tests breaking the pipeline. How do you handle them? . . . 73

Q69. A GitHub Actions workflow is stuck in "pending" or "queued" state. How do you troubleshoot it? . . . 74

Q70. Your team complains of deployment delays due to manual approvals. How would you automate this safely? . . . 75

Q71. Your production pipeline is blocked due to missing approvals and stakeholders are unreachable. What will you do? . . . 76

Q72. A pipeline deployment failed in production. How would you troubleshoot it step by step? . . . 77

Q73. A production deployment caused downtime or an incident. How do you roll back and identify the root cause? . . . 78

Q74. A new deployment causes a CPU spike. How will you roll back and debug? . . . 79

Q75. A deployment to staging failed during the DB-migration step and blocked all environments. What is your rollback plan? . . . 80

Q76. During a blue-green deployment, shifting traffic to green caused a spike in errors. What do you check and fix? . . . 81

Q77. How do you know your last deploy was actually successful? Monitoring? Business metrics? Or silence? . . . 82

Q78. You notice a sudden spike in failed deployments. How would you investigate? . . . 83

Q79. Jenkins is failing to push a Docker image to the registry. How do you troubleshoot? . . . 84

CI/CD, Jenkins, GitOps & Ansible

Basic

Q1. What is the difference between CI, CD and GitOps?

Also asked as: What is the difference between CI and CD? CI vs CD vs GitOps: explain all 3 with real-world examples.

In simple words: CI is like a kitchen that tastes every new ingredient the moment it arrives. CD is the waiter who is always ready to carry the finished dish to the table - either after the manager says "go" (Delivery) or automatically (Deployment). GitOps is a written menu on the wall: a robot keeps checking that what is on every table matches the menu, and to change a dish you change the menu.

Detailed answer

- **Continuous Integration (CI):** every code push is automatically built and tested. Goal: catch bugs within minutes, keep the main branch always working.
- **Continuous Delivery:** every change that passes CI is packaged and deployable to production at any time, but the final push to prod needs a human approval.
- **Continuous Deployment:** same as Delivery, but with no human gate - if all tests pass, it goes to production automatically.
- **GitOps:** a way of doing CD where **Git is the single source of truth** for what should run. It is **pull-based**: an agent inside the cluster (Argo CD / Flux) continuously compares the cluster with Git and **reconciles** differences. Deploy = merge a commit; rollback = revert a commit; manual drift is undone automatically.

Approach	What happens	Who decides prod release
CI	build, unit test, lint, scan on every push	-
Continuous Delivery	package, deploy to staging, prod-ready	Human approval
Continuous Deployment	auto deploy to prod	Pipeline (tests)
GitOps	agent pulls desired state from Git and syncs	Whoever merges the Git PR

Real-world examples (smart-metering style project):

- **CI:** a developer changes the meter-data ingestion service; the PR triggers build, unit tests for the parser of meter readings, SonarQube and Trivy - the PR cannot merge until it is green.
- **Continuous Delivery:** the merged billing (MDMS) service image is deployed to staging automatically; the production release waits for the release manager's approval because it affects customer bills.
- **Continuous Deployment:** an internal Grafana dashboard or alert-rule change goes straight to production once validation passes - low risk, no approval.
- **GitOps:** the head-end (HES) services' Helm values live in a config repo; bumping the image tag in `envs/prod/values.yaml` through a PR makes Argo CD roll it out, and a `git revert` rolls it back.

Example:

```
git push -> Jenkins: build + unit tests (CI)
         -> Docker image pushed to ECR, deployed to staging (CD)
         -> "Approve prod?" button (Continuous Delivery)
         -> or: PR bumps tag in gitops repo -> Argo CD syncs prod (GitOps)
```

> ***What to say to the interviewer:*** *CI means every commit is automatically built and tested so integration problems are caught early. Continuous Delivery keeps a deployable artifact ready with a manual approval before prod, while Continuous Deployment pushes to prod automatically when all checks pass. GitOps is a pull-based way to do CD where Git holds the desired state and an agent like Argo CD reconciles the cluster to it, so deploys and rollbacks are just Git commits.*

Q2. What triggers your pipeline, and how do you trigger a Jenkins job automatically?

Also asked as: How do you trigger a job automatically in Jenkins?

> **In simple words:** A pipeline is like a doorbell-activated machine. Something has to press the bell - a code push, a pull request, a timer, another machine, or a person clicking a button.

Detailed answer

Common triggers:

- **Webhook on push / merge** to a branch (most common) - GitHub/GitLab/Bitbucket calls Jenkins.
- **Pull request opened/updated** - runs CI checks before merge.
- **Tag creation** (e.g. `v1.4.0`) - triggers a release build.
- **Build periodically (cron)** - nightly builds, security scans, cleanup jobs. `H/5 * * * *` = every 5 minutes; `H` spreads jobs so they do not all start at the same second.
- **Poll SCM** - Jenkins checks the repo on a schedule and builds only if there are new commits (fallback when GitHub cannot reach Jenkins).
- **Upstream/downstream** - job B runs after job A succeeds (`upstream` trigger, or `build job:` step from A).
- **Remote trigger** - "Trigger builds remotely" with an authentication token, or the REST API with a user API token, e.g. called from another system.
- **Manual** - "Build with Parameters" for hotfixes or prod deploys.

Note: cron builds even if nothing changed; Poll SCM builds only on new commits.

Example:

```
pipeline {
  agent any
  triggers {
    githubPush()                              // webhook from GitHub
    cron('H 2 * * *')                         // nightly at ~2 AM
    pollSCM('H/5 * * * *')                    // check for commits every ~5 min
    upstream(upstreamProjects: 'build-lib', threshold: hudson.model.Result.SUCCESS)
  }
  stages { stage('Build') { steps { sh 'make build' } } }
}

# Remote trigger (job config: "Trigger builds remotely", token = deploy123)
curl -X POST -u priyanka:$API_TOKEN \
  "https://jenkins.example.com/job/orders/build?token=deploy123"
```

> ***What to say to the interviewer:*** *Our pipelines are mainly triggered by GitHub webhooks on push and pull requests - PRs run CI checks and merges to main trigger build and deploy to dev. Release tags trigger production builds, and we also have cron-triggered nightly scans. Prod deploys additionally need a manual approval.*

Q3. What are webhooks?

> **In simple words:** Polling is you calling the pizza shop every 5 minutes asking "is it ready?". A webhook is the shop calling you when it is ready.

Detailed answer

A webhook is an HTTP POST request that one system sends to another when an event happens. In CI/CD, GitHub sends a webhook to Jenkins when someone pushes code or opens a PR. The payload (JSON) contains the repo, branch, commit ID, and author.

How to set it up:

1. In Jenkins install the GitHub plugin; in the job enable "GitHub hook trigger for GITScm polling".
2. In GitHub repo -> Settings -> Webhooks -> add `https://jenkins.company.com/github-webhook/`, content type JSON, choose events (push, pull request).
3. Add a secret token so Jenkins can verify the request really came from GitHub.

Benefits vs polling: instant builds, less load on Git server and Jenkins.

Example:

```
Payload URL : https://jenkins.example.com/github-webhook/
Content type: application/json
Secret      : <shared secret>
Events      : Push, Pull requests

# Test from GitHub: Settings -> Webhooks -> Recent Deliveries -> Redeliver
```

> ***What to say to the interviewer:*** *A webhook is an event-driven HTTP callback - when code is pushed, GitHub immediately POSTs the event to Jenkins, which starts the build. It is faster and lighter than polling. We secure it with a shared secret and make sure the Jenkins URL is reachable from GitHub.*

Q4. What is a Jenkins agent (worker node)? Where does a pipeline run and where do you define agents?

Also asked as: Where does the Jenkins pipeline run? What is a worker node? Where do you define Jenkins worker nodes?

> **In simple words:** The Jenkins controller is the manager who hands out work. Agents are the workers who actually do the building. The manager should not be doing the heavy lifting.

Detailed answer

- **Controller (master):** stores job config, schedules builds, serves the UI. Ideally runs zero builds itself.
- **Agent (worker node):** a machine, VM, container or Kubernetes pod that connects to the controller and executes the pipeline steps. Each agent has **executors** (parallel build slots) and **labels** (e.g. `linux`, `docker`, `maven`).
- **Where defined:** Manage Jenkins -> Nodes (static agents via SSH or inbound JNLP agent), or Manage Jenkins -> Clouds (dynamic agents from Kubernetes plugin, EC2 plugin, Docker plugin).
- **In the Jenkinsfile** you choose which agent with the `agent` directive.

Example:

```
pipeline {
  agent { label 'docker-linux' }          // run on any agent with this label
  stages {
    stage('Build') { steps { sh 'mvn -B package' } }
  }
}

// Dynamic pod agent on Kubernetes (Kubernetes plugin)
pipeline {
  agent {
    kubernetes {
      yaml '''
        apiVersion: v1
        kind: Pod
        spec:
          containers:
          - name: maven
            image: maven:3.9-eclipse-temurin-17
            command: ["sleep"]
            args: ["infinity"]
      '''
    }
  }
  stages { stage('Build') { steps { container('maven') { sh 'mvn -B package' } } } }
}
```

> ***What to say to the interviewer:*** *The pipeline runs on Jenkins agents, not on the controller. Agents are defined under Manage Jenkins -> Nodes for static machines, or under Clouds for dynamic ones like Kubernetes pods or EC2 instances. In the Jenkinsfile the agent directive with a label or a pod template picks where each stage runs.*

Q5. What is the file structure of a Jenkinsfile, and how do you use it in your project?

Also asked as: How do you write pipeline scripts in Jenkins?

In simple words: A Jenkinsfile is a recipe card kept in the same box as the ingredients (the code). Anyone can read it, change it, and review changes to it.

Detailed answer

A Jenkinsfile is pipeline-as-code, stored in the root of the repository. Two styles: **Declarative** (structured, recommended) and **Scripted** (Groovy, flexible). Declarative building blocks:

- `pipeline` - the outer block.
- `agent` - where to run.
- `environment` - variables and credentials.
- `options` - timeouts, build discarder, timestamps.
- `parameters` - inputs for manual runs.
- `triggers` - cron, webhooks.
- `stages` -> `stage` -> `steps` - the actual work.
- `when` - conditional stages (e.g. only on main).
- `post` - always / success / failure actions (notify, cleanup).

Example:

```
pipeline {
  agent { label 'linux' }
  options { timeout(time: 30, unit: 'MINUTES'); timestamps() }
  environment { APP = 'orders-api' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build')    { steps { sh 'mvn -B -DskipTests package' } }
    stage('Test')     { steps { sh 'mvn test' } }
    stage('Deploy')   {
      when { branch 'main' }
      steps { sh './deploy.sh dev' }
    }
  }
  post {
    failure { echo 'Notify Teams/Slack here' }
    always  { cleanWs() }
  }
}
```

What to say to the interviewer: *We keep a declarative Jenkinsfile in the root of each repo, so the pipeline is versioned and reviewed like code. It has agent, environment, options, stages for checkout, build, test, scan, image push and deploy, when-conditions for branch-specific stages, and a post block for notifications and cleanup.*

Q6. What is a multibranch pipeline?

In simple words: Instead of creating a new pipeline by hand for every branch, Jenkins walks through the repo, finds every branch that has a Jenkinsfile, and creates a pipeline for each automatically.

Detailed answer

- Jenkins scans the repo (on webhook or schedule) and creates one job per branch / PR that contains a Jenkinsfile.
- When a branch is deleted, its job is removed (orphan cleanup).
- Each branch uses its own version of the Jenkinsfile - great for feature branches.
- Inside the Jenkinsfile you use `when { branch 'main' }` or `env.BRANCH_NAME` so feature branches only build/test while main also deploys.

Example:

```
stage('Deploy to Dev') {
  when { branch 'develop' }
  steps { sh './deploy.sh dev' }
}
stage('Deploy to Prod') {
  when { buildingTag() }        // only for release tags like v1.2.0
  steps { sh './deploy.sh prod' }
}
// PR builds: env.CHANGE_ID is set, so we only run build + test
```

> ***What to say to the interviewer:*** *A multibranch pipeline automatically discovers every branch and pull request with a Jenkinsfile and creates a pipeline for each. Feature branches and PRs only run build and tests, while develop deploys to dev and tags deploy to prod, all controlled with when-conditions in one Jenkinsfile.*

Q7. How do you define and invoke pipelines in Jenkins?

> **In simple words:** Defining a pipeline is writing the recipe (Jenkinsfile). Invoking it is deciding what rings the kitchen bell - a code push, a timer, a button, or another recipe finishing.

Detailed answer

Defining:

- **Pipeline script from SCM (recommended):** create a Pipeline job, choose "Pipeline script from SCM", point to the repo and the path `Jenkinsfile`. The pipeline is versioned with the code.
- **Inline script:** pasted into the job UI - only for quick tests, not reviewable.
- **Multibranch Pipeline / Organization Folder:** Jenkins auto-creates jobs for every branch/PR/repo that has a Jenkinsfile.
- **Job DSL / Configuration as Code (JCasC):** create the jobs themselves from code, so even job setup is not click-ops.

Invoking:

- **Triggers:** webhook (`githubPush()`), `cron`, `pollSCM`, `upstream`.
- **Manual:** "Build Now" or "Build with Parameters" (`parameters {}` block).
- **From another pipeline:** `build job: 'deploy-app', parameters: [...]`.
- **Remotely:** REST API with a user API token, or the Jenkins CLI.

Example:

```
pipeline {
  agent any
  parameters {
    choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Target')
    string(name: 'TAG', defaultValue: '', description: 'Image tag to deploy')
  }
  triggers { githubPush() }
  stages {
    stage('Deploy') {
      steps {
        build job: 'deploy-orders', wait: true,
              parameters: [string(name: 'ENV', value: params.ENV),
                           string(name: 'TAG', value: params.TAG)]
      }
    }
  }
}

# Remote trigger via REST API
curl -X POST -u priyanka:$API_TOKEN \
  "https://jenkins.example.com/job/deploy-orders/buildWithParameters?ENV=dev&TAG=1.4.0"
```

> ***What to say to the interviewer:*** *I define pipelines as a Jenkinsfile in the repo and create a Pipeline or Multibranch job that reads it from SCM, with the jobs themselves created through Job DSL or JCasC. They are invoked by GitHub webhooks, cron triggers, manual Build with Parameters, the build job step from another pipeline, or the REST API.*

Q8. What are the types of jobs in Jenkins?

> **In simple words:** Jenkins job types are like different kinds of vehicles - a scooter (Freestyle) for quick simple trips, a truck with a written route (Pipeline), and a whole fleet that auto-assigns a truck to every branch (Multibranch).

Detailed answer

Job type	What it is	When to use
Freestyle	UI-configured steps (shell, build, post-build actions)	Simple one-off tasks; legacy jobs
Pipeline	Stages defined in a Jenkinsfile (inline or from SCM)	Standard CI/CD as code
Multibranch Pipeline	Auto-creates a Pipeline job per branch/PR with a Jenkinsfile	Feature branch + PR builds
Organization Folder	Scans a whole GitHub org/Bitbucket team, creates multibranch jobs per repo	Many repos, zero manual job creation
Folder	Container to group jobs, with folder-scoped credentials and permissions	Separate teams/environments
Multi-configuration (Matrix)	Runs the same build across combinations (OS x JDK)	Compatibility testing (Pipeline `matrix` is the modern way)

Freestyle vs Pipeline: Freestyle config lives in the Jenkins UI (not versioned, hard to review, weak at complex flows); Pipeline is code in Git, supports parallel stages, approvals, resuming and shared libraries.

Example:

```
// Modern matrix instead of a Multi-configuration job
pipeline {
  agent none
  stages {
    stage('Test') {
      matrix {
        axes {
          axis { name 'JDK'; values '11', '17' }
          axis { name 'OS';  values 'linux', 'windows' }
        }
        agent { label "${OS}" }
        stages { stage('Run') { steps { echo "Testing JDK ${JDK} on ${OS}" } } }
      }
    }
  }
}
```

> ***What to say to the interviewer:*** *The main Jenkins job types are Freestyle, Pipeline, Multibranch Pipeline, Organization Folder, Folder, and Multi-configuration or matrix jobs. We mostly use Multibranch Pipelines and Organization Folders so every repo and branch with a Jenkinsfile gets its job automatically, and Folders to scope credentials and permissions per team; Freestyle is only for legacy or trivial tasks.*

Q9. How do you configure GitHub in Jenkins for Freestyle and Pipeline jobs?

> **In simple words:** Jenkins needs two things from GitHub: a key to read the code (credentials) and a doorbell so GitHub can tell Jenkins "new code arrived" (webhook).

Detailed answer

Common setup (once):

1. Install **Git**, **GitHub** and **GitHub Branch Source** plugins.
2. Add credentials: a **GitHub App** (preferred - fine-grained, higher API limits, no personal account) or a **Personal Access Token** as Username/Password, or an SSH key for git clone.
3. Manage Jenkins -> System -> GitHub -> add GitHub Server with the credential, tick "Manage hooks" (optional, lets Jenkins create webhooks).
4. In the GitHub repo: Settings -> Webhooks -> `https://<jenkins>/github-webhook/`, JSON, push + PR events.

Freestyle job:

- Source Code Management -> Git -> repo URL + credentials + branch (`*/main`).
- Build Triggers -> tick **GitHub hook trigger for GITScm polling**.
- Build steps: Execute shell; post-build actions for reports/notifications.

Pipeline job:

- Definition -> **Pipeline script from SCM** -> Git -> repo URL + credentials -> Script Path `Jenkinsfile`.
- Tick "GitHub hook trigger for GITScm polling" (or `triggers { githubPush() }` in the Jenkinsfile).

- Multibranch: Branch Sources -> GitHub -> credential + repo; webhooks trigger re-scans automatically.

Example:

```
// Jenkinsfile checkout using the stored credential
checkout([$class: 'GitSCM',
  branches: [[name: '*/main']],
  userRemoteConfigs: [[url: 'https://github.com/org/orders-api.git',
                       credentialsId: 'github-app-creds']]])

# Verify: GitHub -> Settings -> Webhooks -> Recent Deliveries shows HTTP 200
```

> ***What to say to the interviewer:*** *I install the Git, GitHub and Branch Source plugins, add a GitHub App or PAT credential, and configure a webhook to jenkins-url/github-webhook. For Freestyle I set the Git repo in Source Code Management and tick GitHub hook trigger for GITScm polling; for Pipeline I choose Pipeline script from SCM with the Jenkinsfile path and the same trigger, or use a Multibranch job with a GitHub branch source.*

Q10. How do you know the whole Jenkins pipeline actually executed successfully?

> **In simple words:** Like tracking a courier parcel - you look at every checkpoint scan (stages), the final "delivered" status, the photo proof (reports and artifacts), and you get an SMS at the end (notification).

Detailed answer

Evidence to check:

- **Build status/result:** SUCCESS, UNSTABLE (tests failed but build continued), FAILURE, ABORTED - a green ball is not enough if stages were skipped.
- **Stage View / Blue Ocean / Pipeline Graph View:** each stage shows passed, failed, or **skipped** (grey) - a skipped deploy stage means it did not run.
- **Console Output:** full log with timestamps (`timestamps{ }` option); Pipeline Steps view shows each step and its duration.
- **Test reports:** `junit` step shows test count and failures; coverage reports.
- **Artifacts / fingerprints:** `archiveArtifacts`, image tag in ECR, Helm release revision.
- **post blocks:** `always` / `success` / `failure` / `unstable` send notifications and publish results.
- **Notifications:** Teams/Slack/email with job, build number, commit and deployed version.
- **Deployment verification:** the pipeline itself runs `kubectl rollout status` or a smoke test, so "success" means the app is really healthy, not just "command ran".
- **Audit:** build history, Audit Trail plugin, who approved the input step.

Example:

```
post {
  success {
    office365ConnectorSend webhookUrl: env.TEAMS_HOOK,
      message: "SUCCESS ${env.JOB_NAME} #${env.BUILD_NUMBER} deployed ${env.TAG}"
  }
  unstable { echo 'Tests failed - check JUnit report' }
  failure  { echo "Failed at stage: ${env.STAGE_NAME}" }
  always   { junit 'reports/*.xml'; archiveArtifacts 'reports/**' }
}
stage('Verify') { steps { sh 'kubectl -n prod rollout status deploy/orders-api --timeout=5m' } }
```

What to say to the interviewer: *I check the build result and the Stage View or Blue Ocean to confirm every stage ran and none was skipped, the console output and step timings, published JUnit and coverage reports, and archived artifacts or the pushed image tag. Post blocks send Teams or Slack notifications for success and failure, and the pipeline ends with a rollout status or smoke test so success really means the app is healthy.*

Q11. What types of artifacts are generated in your pipeline?

In simple words: An artifact is the finished product that comes off the assembly line - the thing you actually ship, not the raw code.

Detailed answer

Typical artifacts:

- **Docker images** pushed to ECR / Docker Hub / Artifactory, tagged with version + git SHA.
- **Helm charts** (packaged `.tgz`) pushed to a chart repo or OCI registry.
- **Binaries/packages:** `.jar`/`.war` (Java), `.zip` (Lambda), `.whl`, npm packages - stored in Nexus/Artifactory/S3.
- **Reports:** unit test reports (JUnit XML), code coverage, SonarQube results, Trivy scan reports, SBOM.
- **IaC outputs:** saved `terraform plan` files.

Key rule: **build once, deploy the same artifact everywhere** - dev, staging and prod all use the exact same image tag.

Example:

```
IMAGE=123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-api
TAG=1.4.0-${GIT_COMMIT:0:7}          # e.g. 1.4.0-a1b2c3d
docker build -t $IMAGE:$TAG .
docker push $IMAGE:$TAG
helm package charts/orders-api --version 1.4.0
archiveArtifacts artifacts: 'target/*.jar, reports/**', fingerprint: true
```

What to say to the interviewer: *Our main artifacts are Docker images pushed to ECR, tagged with the release version and short git commit SHA, plus Helm charts, test and coverage reports, and security scan reports. We build the artifact once and promote the same immutable tag from dev to prod.*

Q12. What is Blue-Green deployment vs Canary deployment? Explain with examples.

Also asked as: What is Blue-Green deployment? What is Canary deployment? Explain Blue-Green vs Canary with examples. Blue-green: how do you shift traffic safely at 10%, 25%?

In simple words: Blue-Green is having two identical stages in a theatre - the show runs on Blue while you set up Green, then you swing the spotlight to Green in one go. Canary is letting a few people taste a new dish first; if they are fine, you serve it to everyone.

Detailed answer

Blue-Green:

- Two full environments: Blue (current live) and Green (new version).
- Deploy and test Green, then switch all traffic (load balancer / DNS / K8s Service selector) at once.
- Rollback = switch back to Blue instantly.
- Cost: double infrastructure for the switch window.

Canary:

- Release new version to a small slice of traffic (1% -> 10% -> 50% -> 100%).
- Watch error rate, latency, CPU at each step; auto-rollback if metrics get worse.
- Cheaper, lower blast radius, but slower and needs good monitoring.

How to shift traffic safely in steps (10% -> 25% -> 50% -> 100%):

- **ALB weighted target groups:** one listener rule forwards to blue and green target groups with weights (90/10, then 75/25...). Instant, no DNS caching.
- **Route 53 weighted records:** two records with weights; simple, but clients and resolvers cache DNS for the TTL, so shifts are slower and less precise.
- **Argo Rollouts (Kubernetes):** `setWeight` steps with `pause` and an **analysis** step between them that checks Prometheus metrics.
- **CodeDeploy (ECS/Lambda):** built-in configs like `CodeDeployDefault.ECSCanary10Percent5Minutes` (10% for 5 minutes, then 100%) or `ECSLinear10PercentEvery1Minutes`.
- **Automatic rollback:** tie each step to alarms (5xx rate, p99 latency). CodeDeploy rolls back when a CloudWatch alarm fires; Argo Rollouts aborts when analysis fails. Hold at each step long enough to see real traffic.

	Blue-Green	Canary
Traffic shift	All at once	Gradual %
Rollback	Instant switch back	Shift weight to 0%
Infra cost	2x during release	Small extra
Needs	Good pre-switch tests	Strong metrics / monitoring
Best for	DB-safe big releases, fast rollback	High-traffic user-facing apps

Example:

```
# Blue-Green on Kubernetes: flip the Service selector
kubectl patch service orders -p '{"spec":{"selector":{"app":"orders","version":"green"}}}'

# Canary with AWS ALB weighted target groups (90/10)
aws elbv2 modify-listener --listener-arn $LISTENER --default-actions \
 '[{"Type":"forward","ForwardConfig":{"TargetGroups":[
   {"TargetGroupArn":"'$TG_V1'","Weight":90},{"TargetGroupArn":"'$TG_V2'","Weight":10}]}}]'

# Argo Rollouts: 10% -> 25% -> 50% -> 100% with pauses and metric analysis
strategy:
  canary:
    steps:
    - setWeight: 10
    - pause: { duration: 5m }
    - analysis: { templates: [ { templateName: error-rate } ] }
    - setWeight: 25
    - pause: { duration: 10m }
    - setWeight: 50
    - pause: { duration: 10m }
```

What to say to the interviewer: *Blue-Green runs two identical environments and switches all traffic to the new one after testing, so rollback is an instant switch back but it costs double infrastructure briefly. Canary sends a small percentage of traffic to the new version and increases it while watching error rate and latency, which limits blast radius. I use Blue-Green when I need instant rollback and Canary for high-traffic services with good monitoring.*

Q13. How do you handle build failures?

In simple words: When the oven beeps "failed", you do not just press start again - you read the error, see which step burned, fix the cause, and make sure the cook gets told right away.

Detailed answer

1. **Fail fast and notify:** pipeline stops at the failing stage; `post { failure {...} }` sends a Teams/Slack/email message with the build link and commit author.
2. **Read the console log** of the failed stage - compilation error, failed test, dependency download, docker push, timeout.
3. **Classify:** code problem (dev fixes), test flakiness (quarantine + ticket), infra problem (agent disk full, network, credentials expired - DevOps fixes).
4. **Reproduce locally** with the same Docker build image so "works on my machine" is ruled out.
5. **Protect main:** branch protection requires a green build before merge, so failures stay on feature branches.
6. **Keep evidence:** archive test reports and logs; use `retry` only for known transient network steps, never to hide real failures.

Example:

```
post {
  failure {
    office365ConnectorSend webhookUrl: env.TEAMS_HOOK,
      message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} (${env.GIT_COMMIT})"
  }
  always { junit 'target/surefire-reports/*.xml' }
}
stage('Download deps') {
  steps { retry(3) { sh 'mvn -B dependency:go-offline' } }   // transient network only
}
```

> ***What to say to the interviewer:*** *The pipeline fails fast and notifies the team with the build link. I check the failing stage log, classify it as code, test or infrastructure, and route it to the right owner. Branch protection keeps main green, and I only use retries for known transient network steps, never to hide real failures.*

Q14. What branching strategy do you follow, and how do you handle a production bug?

> **In simple words:** Branches are separate drafts of a document. Trunk-based means everyone edits one main draft in small pieces. GitFlow means several long-lived drafts (develop, release, main). A hotfix is an emergency patch made on a copy of exactly what customers have right now.

Detailed answer

	GitFlow	Trunk-based
Long-lived branches	main, develop, release/*, hotfix/*	only main
Feature branches	long-ish	short (hours to 1-2 days)
Release style	scheduled releases	continuous, feature flags hide unfinished work
Good for	versioned products, slower release cycles	microservices, frequent deploys

Rules we apply either way:

- **Protected main:** no direct push, PR with at least 1-2 reviews, required green CI checks (build, tests, scans).
- **PR checks:** lint, unit tests, coverage gate, SonarQube, secret scan.
- **Releases tagged:** `v1.4.0` tag = exactly what is in prod.

Handling a production bug (hotfix):

1. Create `hotfix/1.4.1` from the **prod tag** (`v1.4.0`), not from develop, so no unreleased code sneaks in.
2. Fix + test; PR with fast-track review; full pipeline still runs.
3. Tag `v1.4.1`, deploy to prod (via staging smoke test).
4. **Merge the fix back** to main and develop (or cherry-pick) so the bug does not return in the next release.
5. RCA and add a test that would have caught it.

Example:

```
git checkout -b hotfix/1.4.1 v1.4.0
# fix the bug, commit
git push origin hotfix/1.4.1          # PR -> review -> CI green
git tag v1.4.1 && git push origin v1.4.1   # tag triggers prod release pipeline
git checkout develop && git merge --no-ff hotfix/1.4.1
```

> ***What to say to the interviewer:*** *For microservices I prefer trunk-based development with short-lived feature branches, protected main, mandatory PR reviews and required CI checks, while GitFlow suits scheduled versioned releases. For a production bug I branch a hotfix from the production tag, fix and test it through the pipeline, tag and release it, then merge it back to main and develop so it is not lost, followed by an RCA and a regression test.*

Q15. What is the difference between git merge and git rebase? When do you use each?

Also asked as: Describe the Git merge process. How do you resolve merge conflicts?

> **In simple words:** Merge is like tying two ropes together with a visible knot - you can always see where they joined. Rebase is like cutting your rope and re-attaching it at the end of the other rope so it looks like one straight line.

Detailed answer

- **git merge:** combines two branches and creates a **merge commit**. History is preserved exactly as it happened (non-linear). Safe for shared branches.
- **git rebase:** replays your commits on top of the latest target branch, creating **new commits (new SHAs)**. History becomes linear and clean.
- **Squash merge:** all commits of a feature branch become **one commit** on main - clean history, one commit per PR, easy to revert.

History shape:

```
Before:           A---B---C  main
                       \
                        D---E  feature

After merge:      A---B---C-------M  main      (M = merge commit)
                       \         /
                        D-------E

After rebase:     A---B---C---D'---E'  feature  (D', E' are new commits)

After squash:     A---B---C---S  main          (S = D+E in one commit)
```

When to use:

- **Rebase** your own local/feature branch onto main before opening or updating a PR, to get a clean linear history and resolve conflicts yourself.
- **Merge** (or squash merge) to bring a finished PR into main.
- **Golden rule: never rebase a shared/public branch** (main, develop, a branch others pulled) - it rewrites history and breaks everyone else's copy. If you must update a rebased branch on the remote, use `git push --force-with-lease`, never plain `--force`.

Example:

```
# Update my feature branch with latest main (clean linear history)
git checkout feature/login
git fetch origin
git rebase origin/main          # fix conflicts -> git add . -> git rebase --continue
git push --force-with-lease     # only because it is MY branch

# Merge a finished branch into main
git checkout main
git merge --no-ff feature/login

# Squash merge
git merge --squash feature/login && git commit -m "Add login feature"
```

Resolving merge conflicts step by step:

1. `git merge feature/x` (or a rebase) stops with `CONFLICT (content): Merge conflict in app.yaml`.
2. `git status` lists the files marked "both modified".
3. Open each file and find the markers: `<<<<<<< HEAD` (your side), `=======`, `>>>>>>> feature/x` (their side). Edit to the correct final content and delete the markers.
4. `git add <file>` to mark it resolved, then `git commit` (or `git rebase --continue` during a rebase).
5. Stuck? `git merge --abort` or `git rebase --abort` returns to the state before you started.
6. Helpers: `git mergetool`, VS Code's merge editor, `git diff --name-only --diff-filter=U` to list unresolved files. Run the tests before pushing.

Prevention: small, short-lived branches, pull or rebase from main often, and CODEOWNERS on hot files.

```
git merge feature/retry
# CONFLICT (content): Merge conflict in config/app.yaml
git status
#   both modified:   config/app.yaml
# edit the file, remove <<<<<<< ======= >>>>>>> markers
git add config/app.yaml
git commit -m "Merge feature/retry, resolve config conflict"
```

> ***What to say to the interviewer:*** *Merge joins branches with a merge commit and keeps the true history, while rebase replays my commits on top of the target branch to give a clean linear history but rewrites commit IDs. I rebase my own feature branch onto main before a PR, and merge or squash-merge the PR into main. I never rebase shared branches, and when I update my own rebased branch I use force-with-lease.*

Q16. What is git stash used for?

> **In simple words:** Stash is a drawer where you quickly put your half-finished work so you have a clean desk for an urgent job. When you are done, you take your work back out of the drawer.

Detailed answer

- `git stash` saves uncommitted changes (staged and unstaged) and cleans the working directory, without making a commit.

- `git stash push -m "message"` - stash with a name so you remember what it is.
- `git stash list` - see all stashes (`stash@{0}` is the newest).
- `git stash apply` - restore changes but **keep** the stash (useful to apply to several branches).
- `git stash pop` - restore changes and **delete** the stash.
- `git stash -u` (`--include-untracked`) - also stash new untracked files; by default they are left behind.
- `git stash show -p stash@{1}` - see what is inside; `git stash drop` / `clear` to remove.

Real use case: I am halfway through a feature, and a production bug needs a hotfix now. I stash my work, switch to a hotfix branch, fix and push, then come back and pop my stash.

Example:

```
git stash push -u -m "WIP: login page"      # save work incl. new files
git checkout -b hotfix/1.4.1 v1.4.0          # clean tree -> do the urgent fix
# ... fix, commit, push ...
git checkout feature/login
git stash list                               # stash@{0}: On feature/login: WIP: login page
git stash pop                                # changes back, stash removed
```

> ***What to say to the interviewer:*** *Git stash temporarily saves uncommitted changes and gives me a clean working tree without committing. I use stash push with a message and -u to include untracked files, list to see stashes, apply to restore while keeping the stash, and pop to restore and remove it. A typical case is stashing half-done feature work to switch branches for an urgent hotfix, then popping it back afterwards.*

Q17. How do you store credentials securely in Jenkins?

Also asked as: How do you handle secrets in Jenkins pipelines?

> **In simple words:** Never write the house key on the door. Keep it in a locked key box (Jenkins Credentials store or a vault) and hand it to the worker only while they are working, then take it back.

Detailed answer

- Use the **Jenkins Credentials** store (Manage Jenkins -> Credentials): types like Username/Password, Secret text, SSH key, Secret file, AWS credentials. They are encrypted on disk.
- Scope them to a **folder** so only relevant jobs can use them.
- Access them in pipelines with `withCredentials` or `credentials()` - Jenkins masks the values in logs.
- Better: don't store long-lived cloud keys at all - use **IAM roles** on agents (EC2 instance profile / EKS IRSA) or fetch secrets at runtime from **HashiCorp Vault / AWS Secrets Manager** plugins.
- Restrict who can view/configure credentials with Role-Based Access (Role Strategy / Matrix Authorization).
- Never `echo` secrets, never put them in the Jenkinsfile or Git.

Example:

```
pipeline {
  agent any
  environment { SONAR_TOKEN = credentials('sonar-token') }   // masked in logs
  stages {
    stage('Login') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'nexus-creds',
                         usernameVariable: 'U', passwordVariable: 'P')]) {
          sh 'echo "$P" | docker login nexus.example.com -u "$U" --password-stdin'
        }
      }
    }
  }
}
```

> ***What to say to the interviewer:*** *I store secrets in the Jenkins Credentials store, scoped per folder, and use them via withCredentials so they are masked in logs. For AWS I avoid static keys entirely and use IAM roles on the agents, and for application secrets we pull from Vault or Secrets Manager at runtime.*

Q18. Azure DevOps: How do you enforce branch policies and PR validation in Azure Repos?

> **In simple words:** Branch policies are the rules at the door of the main branch: nobody walks in alone, someone must check your work, the robot must test it, and only a few trusted people have a key to skip the line.

Detailed answer

Set on the branch (Repos -> Branches -> main -> Branch policies), or across many repos at project level:

- **Require a minimum number of reviewers** (e.g. 2); optionally "reset votes when new changes are pushed" and "prohibit the most recent pusher from approving their own changes".
- **Check for linked work items:** every PR must link a Boards item (traceability).
- **Check for comment resolution:** all review comments must be resolved before merge.
- **Build validation:** a pipeline runs on every PR (build + tests + scans); the PR cannot complete unless it passes. Can be required or optional, with path filters, and expires after main changes.
- **Status checks:** external services (SonarQube, security scanners) post a status that must succeed.
- **Automatically included reviewers by path:** e.g. `/infra/*` requires the platform team, `/db/migrations/*` requires a DBA.
- **Limit merge types:** allow only squash merge (or rebase) to keep history clean.
- **Limit who can bypass:** the "Bypass policies when completing pull requests" and "Bypass policies when pushing" permissions should be given only to a small admin group, and their use audited. Direct pushes to main are blocked once policies are on.

Example:

```
# Build validation pipeline triggered by the PR policy (azure-pipelines-pr.yml)
trigger: none
pr: none                      # Azure Repos PRs are triggered by the branch policy
pool: { vmImage: ubuntu-latest }
steps:
- script: npm ci && npm run lint && npm test
- task: SonarQubePrepare@6
  inputs: { SonarQube: 'sonar-sc', scannerMode: 'CLI' }

# Add a required reviewer policy with the CLI
az repos policy required-reviewer create --repository-id $REPO_ID --branch main \
  --blocking true --enabled true --path-filter "/infra/*" \
  --required-reviewer-ids platform-team@company.com --message "Infra change"
```

What to say to the interviewer: *I protect main with branch policies: minimum two reviewers with votes reset on new pushes, linked work items, all comments resolved, and a required build validation pipeline that runs build, tests and scans on every PR. External status checks like SonarQube, path-based required reviewers for infra or DB changes, squash-only merges, and very limited, audited bypass permissions complete it, so nothing reaches main without review and a green pipeline.*

Q19. Azure DevOps: What is the purpose of YAML pipeline templates?

In simple words: A template is a form with blanks. The platform team designs the form once; every team fills in their own details. With an `extends` template, teams can only fill in the blanks - they cannot remove the security checks printed on the form.

Detailed answer

- **Reuse and consistency:** write build/test/deploy logic once, use it in every pipeline; fix once, fixed everywhere.
- **Types of templates: stage**, **job**, **step** and **variable** templates, included with `- template: path.yml`.
- **Parameters:** typed inputs (`string`, `boolean`, `object`, `stepList`) with defaults and allowed values, so a template can serve many services and environments.
- `extends` **templates for security:** the pipeline must extend an approved template; the template decides the structure (mandatory scans, approved tasks only) and can wrap the user's steps. Combine with the "Required template" check on environments or service connections so only pipelines extending it can deploy.
- **Central template repo:** keep templates in a separate repo, referenced via `resources: repositories` and pinned to a tag or branch (`ref: refs/tags/v2`).
- Equivalents: Jenkins **shared libraries**, GitHub Actions **reusable workflows** and composite actions, GitLab `include`.

Example:

```
# templates repo: deploy-stage.yml
parameters:
- name: env
  type: string
  values: [dev, staging, prod]
- name: service
  type: string
stages:
- stage: deploy_${{ parameters.env }}
  jobs:
  - deployment: deploy
    environment: ${{ parameters.env }}          # approvals and checks live here
    strategy:
      runOnce:
        deploy:
          steps:
          - script: helm upgrade --install ${{ parameters.service }} chart/ -f values-${{ parameters
    .env }}.yaml

# service repo: azure-pipelines.yml
resources:
  repositories:
  - repository: templates
    type: git
    name: Platform/pipeline-templates
    ref: refs/tags/v2
extends:
  template: secure-pipeline.yml@templates      # enforced structure with mandatory scans
  parameters:
    service: orders
```

What to say to the interviewer: *YAML templates let us define stages, jobs, steps and variables once and reuse them with typed parameters across all services and environments. We keep them in a central repo referenced through resources repositories and pinned to a tag, and use extends templates plus the required-template check so every production pipeline must include our security scans. It is the Azure DevOps equivalent of Jenkins shared libraries or GitHub reusable workflows.*

Q20. Ansible: What is an Ansible playbook?

In simple words: A playbook is a written checklist: "on these servers, install nginx, copy this config, start the service". Ansible reads the checklist and does it on 1 or 1000 servers.

Detailed answer

- A playbook is a **YAML** file with one or more **plays**.
- Each play targets `hosts` (from inventory) and runs a list of **tasks**.
- Each task calls a **module** (`apt`, `yum`, `copy`, `template`, `service`, `user`).
- Tasks are **idempotent** - running twice does not change anything if the system is already in the desired state.
- Ansible is **agentless**: it connects over SSH (WinRM for Windows).

Example:

```
- name: Install and start nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
        update_cache: true
    - name: Ensure nginx is running
      service:
        name: nginx
        state: started
        enabled: true

# Run it
ansible-playbook -i inventory.ini site.yml
```

What to say to the interviewer: *A playbook is a YAML file of plays, where each play targets a group of hosts and runs idempotent tasks using modules like apt, copy, template and service. Ansible is agentless and connects over SSH, so we can configure many servers consistently with one command.*

Q21. Ansible: What is the difference between push and pull configuration management?

In simple words: Push is the teacher walking to every desk and handing out the worksheet. Pull is every student walking to the teacher's desk every 30 minutes to pick up the latest worksheet.

Detailed answer

	Push	Pull
Who starts	Central control node	Each managed node
Agent needed	No (SSH)	Yes (agent on node)
Tools	Ansible (default), Salt SSH	Puppet, Chef, ansible-pull
Timing	On demand, immediate	Periodic (e.g. every 30 min)
Good for	Ad-hoc changes, deployments, small/medium fleets	Very large fleets, self-healing drift correction

- Push gives immediate control and simple setup, but the control node must reach every server.
- Pull scales well and auto-corrects drift, but changes are not instant and you must run agents.
- Ansible can also do pull with `ansible-pull` (node clones a Git repo and applies it via cron).

Example:

```
# Push (Ansible default)
ansible-playbook -i inventory/prod site.yml

# Pull (run on each node via cron)
*/30 * * * * ansible-pull -U https://github.com/org/config-repo.git local.yml
```

What to say to the interviewer: *In push mode a central node connects out to servers and applies config immediately - Ansible works this way over SSH with no agent. In pull mode each server runs*

an agent that periodically fetches and applies config, like Puppet or Chef, which scales well and auto-fixes drift. Ansible also supports pull via ansible-pull.

Q22. Ansible: How do you manage Ansible inventory?

In simple words: The inventory is Ansible's phone book - the list of servers, grouped by role, with any special details about each.

Detailed answer

- **Static inventory:** INI or YAML file listing hosts and groups (`[web]`, `[db]`), plus variables.
- **Separate per environment:** `inventory/dev`, `inventory/prod` - you pick with `-i`, so you never hit prod by accident.
- **group_vars / host_vars:** folders holding variables per group or host.
- **Dynamic inventory:** for cloud where IPs change - plugins like `amazon.aws.aws_ec2` query AWS and group hosts by tags.

Example:

```
# inventory/prod/hosts.ini
[web]
web1.prod.internal
web2.prod.internal
[db]
db1.prod.internal ansible_user=ubuntu

# inventory/prod/aws_ec2.yml  (dynamic)
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters:
  tag:Env: prod
keyed_groups:
  - key: tags.Role
    prefix: role

ansible-inventory -i inventory/prod/aws_ec2.yml --graph
```

What to say to the interviewer: *We keep separate inventories per environment with group_vars and host_vars for variables. For AWS we use the aws_ec2 dynamic inventory plugin that discovers instances by tags, so new autoscaled servers are picked up automatically without editing files.*

Q23. Ansible: How do you work with modules and roles?

In simple words: A module is a single tool (a screwdriver). A role is a complete toolkit for one job ("set up nginx") that you can reuse in any project.

Detailed answer

- **Modules** are the units of work: `apt`, `copy`, `template`, `service`, `file`, `user`, `ec2_instance`, etc. Prefer modules over `shell` because they are idempotent.
- **Roles** package tasks, handlers, templates, files, defaults and vars in a standard folder structure so they can be reused and shared (Ansible Galaxy).
- Create a skeleton with `ansible-galaxy init`.

Example:

```
ansible-galaxy init roles/nginx

roles/nginx/
  tasks/main.yml        # what to do
  handlers/main.yml     # restart nginx on change
  templates/nginx.conf.j2
  defaults/main.yml     # default variables (lowest priority)
  vars/main.yml
  files/

# site.yml
- hosts: web
  become: true
  roles:
    - common
    - nginx
```

> ***What to say to the interviewer:*** *Modules are idempotent building blocks like apt, template and service, and I avoid raw shell commands where a module exists. I organize reusable logic into roles created with ansible-galaxy init, each with tasks, handlers, templates and defaults, and combine roles in a site playbook per environment.*

Q24. Ansible: How do you use handlers in a playbook?

> **In simple words:** A handler is "only restart the machine if you actually changed its settings". If nothing changed, nobody gets disturbed.

Detailed answer

- A handler is a special task that runs only when another task **notifies** it and that task reported `changed`.
- Handlers run **once at the end of the play**, even if notified many times.
- Typical use: restart/reload a service after its config file changes.
- Use `meta: flush_handlers` if you need them to run immediately.

Example:

```
- hosts: web
  become: true
  tasks:
    - name: Deploy nginx config
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Reload nginx
  handlers:
    - name: Reload nginx
      service:
        name: nginx
        state: reloaded
```

> ***What to say to the interviewer:*** *Handlers are tasks triggered by notify only when a task actually changes something, and they run once at the end of the play. I use them to reload or restart services after a config template changes, so services are not restarted unnecessarily.*

Q25. Ansible: How do you store secrets securely in Ansible?

In simple words: Ansible Vault is a locked diary - the file sits in Git, but only people with the key can read what is inside.

Detailed answer

- **Ansible Vault** encrypts files or single variables with AES256.
- Store the vault password outside Git - in a CI credential, or a password file/script with restricted permissions.
- Use separate vault IDs per environment (`dev`, `prod`).
- Add `no_log: true` to tasks that handle secrets so they do not appear in output.
- Alternative: fetch secrets at runtime from AWS Secrets Manager or HashiCorp Vault via lookup plugins.

Example:

```
ansible-vault create group_vars/prod/vault.yml
ansible-vault encrypt_string 'S3cr3t!' --name 'db_password'
ansible-playbook site.yml --vault-id prod@~/.vault_pass_prod

# Runtime lookup from AWS Secrets Manager
db_password: "{{ lookup('amazon.aws.aws_secret', 'prod/db/password') }}"

- name: Create DB user
  mysql_user: name=app password="{{ db_password }}"
  no_log: true
```

What to say to the interviewer: *I encrypt secrets with Ansible Vault, keeping the vault password in the CI credential store rather than Git, with separate vault IDs per environment. For sensitive tasks I set no_log, and where possible I look up secrets at runtime from AWS Secrets Manager or HashiCorp Vault.*

Q26. Ansible: How do you run your Ansible projects?

In simple words: Like a well-organized cookbook: recipes (roles) in one folder, the guest list (inventory) per event, and a practice run (check mode) before the real dinner.

Detailed answer

Typical flow:

1. Project layout: `ansible.cfg`, `inventory/<env>`, `group_vars`, `roles/`, `site.yml`, `requirements.yml`.
2. Install dependencies: `ansible-galaxy install -r requirements.yml`.
3. Lint: `ansible-lint`.
4. Dry run: `--check --diff` to preview changes.
5. Limit blast radius: `--limit` one host/group first, `serial` for rolling batches.
6. Run from CI (Jenkins/GitHub Actions) or AWX/Ansible Tower for audit logs and RBAC, not from someone's laptop.

Example:

```
ansible-lint site.yml
ansible-playbook -i inventory/prod site.yml --check --diff
ansible-playbook -i inventory/prod site.yml --limit web1.prod.internal
ansible-playbook -i inventory/prod site.yml --tags nginx

# Rolling update: 25% of hosts at a time
- hosts: web
  serial: "25%"
  max_fail_percentage: 10
```

> ***What to say to the interviewer:*** *Our Ansible repo has per-environment inventories, group_vars and reusable roles. Changes go through ansible-lint and a check-and-diff dry run, then run first on a limited host, then in rolling batches with serial. Runs are triggered from the CI pipeline so they are logged and repeatable.*

Q27. What is the difference between Scripted and Declarative Jenkins pipelines, and when would you use each?

> **In simple words:** Declarative is a form with fixed boxes to fill in (stages, steps, post actions), so it's easy for everyone to read. Scripted is a blank sheet where you write Groovy code, which is powerful but easy to make messy.

Detailed answer

	Declarative	Scripted
Syntax	`pipeline { agent ... stages { stage { steps } } }`	`node { stage('x') { ... } }` plain Groovy
Structure	Fixed, validated before running	Free-form code
Readability	High, easy for the whole team	Depends on the author
Built-ins	`post`, `when`, `options`, `environment`, `parallel`, `input`	Write it yourself with try/catch/finally
Flexibility	Limited, extend with `script {}` blocks	Full Groovy (loops, dynamic stages)
Restart from stage	Supported	Not supported

- **Use Declarative by default** for almost everything; it is the recommended style.
- Use a `script { }` block, or shared-library code, for the few places that need loops or complex logic.
- Use fully **Scripted** only for highly dynamic pipelines (for example stages generated from a list of services).

Example:

```
// Declarative
pipeline {
  agent { label 'docker' }
  stages {
    stage('Build') { steps { sh 'make build' } }
    stage('Test')  { when { branch 'main' } steps { sh 'make test' } }
  }
  post { failure { echo 'notify team' } }
}

// Scripted
node('docker') {
  try {
    stage('Build') { sh 'make build' }
    for (svc in ['api', 'worker']) { stage("Test ${svc}") { sh "make test-${svc}" } }
  } catch (err) { echo 'notify team'; throw err }
}
```

What to say to the interviewer: *Declarative pipelines have a fixed, validated structure with built-in* `when`*,* `post`*,* `options` *and* `parallel`*, so they're easier to read and maintain. Scripted pipelines are plain Groovy and more flexible. I use Declarative by default and drop into a* `script` *block or a shared library only where I need loops or dynamic logic.*

Q28. How do you set up Jenkins from scratch on a fresh server?

In simple words: Setting up a new factory: install the power supply (Java), install the machine (Jenkins), lock the doors (admin user, TLS), and hire workers (agents) so the manager's office isn't used as the shop floor.

Detailed answer

1. **Prerequisites:** a Linux VM (2+ vCPU, 4GB+ RAM, a separate disk for `/var/lib/jenkins`) and Java 17 or 21.
2. **Install:** from the official Jenkins apt/yum repository (LTS), or run the `jenkins/jenkins:lts` container with a persistent volume.
3. **Unlock:** read the initial admin password from `/var/lib/jenkins/secrets/initialAdminPassword`, install the suggested plugins, and create an admin user.
4. **Secure it:** put Nginx in front with TLS, set the Jenkins URL, enable security (matrix or role-based authorization, SSO if possible), and set the controller's executors to 0.
5. **Agents:** add build agents (SSH agents, Docker, or the Kubernetes plugin for ephemeral pods), so builds never run on the controller.
6. **Credentials and tooling:** add Git/registry/cloud credentials in the credentials store, and configure tools (JDK, Maven, Node) or use container agents.
7. **Configuration as Code:** capture settings in JCasC YAML and the plugin list in `plugins.txt`, so the server can be rebuilt.
8. **Backups and monitoring:** back up `JENKINS_HOME` (thinBackup or EBS snapshots), and add the Prometheus metrics plugin.

Example:

```
# Ubuntu, LTS repo
sudo apt update && sudo apt install -y fontconfig openjdk-17-jre
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword

# or Docker
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk17
```

What to say to the interviewer: *I install Java 17 and Jenkins LTS from the official repo or run the LTS container with a persistent volume, unlock it with the initial admin password, install the suggested plugins and create an admin user. Then I secure it with TLS behind Nginx and role-based access, set the controller to zero executors, and add agents. Finally I capture the setup with JCasC and a plugin list so the server can be rebuilt, and back up JENKINS_HOME.*

Q29. How do you install Jenkins plugins, and which plugins are most common in a CI/CD setup?

In simple words: Jenkins on its own is a bare phone; plugins are the apps. Install only what you need and keep them updated, because too many apps slow the phone down and break after updates.

Detailed answer

How to install:

- UI: **Manage Jenkins -> Plugins -> Available**, search, install, and restart if needed.
- CLI or immutable images: list plugins in `plugins.txt` and run `jenkins-plugin-cli --plugin-file plugins.txt` in the Dockerfile. This is repeatable and version-pinned.
- Offline: upload the `.hpi` file under Advanced settings.
- Update carefully: check compatibility, test in staging, and back up `JENKINS_HOME` first, because plugin updates are the most common cause of broken Jenkins servers.

Common plugins:

Purpose	Plugin
Pipelines	Pipeline (workflow-aggregator), Pipeline: Stage View
Source control	Git, GitHub Branch Source, Bitbucket Branch Source
Credentials	Credentials Binding, SSH Agent, AWS Credentials
Containers and K8s	Docker Pipeline, Kubernetes
Quality	JUnit, SonarQube Scanner, Warnings Next Generation
Notifications	Slack Notification, Office 365 Connector, Email Extension
Security and admin	Role-based Authorization Strategy, Configuration as Code
Usability	Timestamper, AnsiColor, Blue Ocean (legacy UI)

Example:

```
FROM jenkins/jenkins:lts-jdk17
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt
# plugins.txt:
#   workflow-aggregator:latest
#   git:latest
#   kubernetes:latest
#   configuration-as-code:latest
```

What to say to the interviewer: *For ad-hoc installs I use Manage Jenkins -> Plugins, but for production I bake plugins into the image with* `jenkins-plugin-cli` *and a pinned* `plugins.txt`*, so the setup is repeatable. The common ones are Pipeline, Git and GitHub Branch Source, Credentials Binding, Docker Pipeline, Kubernetes, JUnit, SonarQube, Slack or Teams notifications, Role-based Authorization and JCasC. I test plugin updates in staging, because they're the most common cause of breakage.*

Q30. Git: what is the "detached HEAD" state, what causes it, and how do you fix it?

In simple words: Normally you stand on a branch, a moving walkway, and new commits move it forward. In detached HEAD you've stepped off onto one specific tile (a commit). You can still walk around and make changes, but nothing is carrying them forward, so they're easy to lose.

Detailed answer

- **Causes:** `git checkout <commit-sha>`, `git checkout v1.2.0` (a tag), `git checkout origin/main` (a remote-tracking branch), during an interactive rebase or bisect. CI systems also check out a specific commit, which is why build logs often show it.
- **It's not an error.** It's useful for inspecting old code or building a tag.
- **To keep work done there:** create a branch from it: `git switch -c fix/from-old-commit`.
- **To just go back:** `git switch main`.
- **Lost the commits after switching away?** `git reflog` shows where HEAD has been, so you can create a branch from the lost SHA.

Example:

```
git checkout v2.3.0
# You are in 'detached HEAD' state...
git status              # HEAD detached at v2.3.0

# made a hotfix commit here and want to keep it:
git switch -c hotfix/2.3.1

# already switched away and lost it?
git reflog              # find: a1b2c3d HEAD@{2}: commit: fix null check
git branch recovered-fix a1b2c3d
```

What to say to the interviewer: *Detached HEAD means HEAD points directly at a commit instead of a branch, usually because I checked out a tag, a SHA or a remote branch, or CI checked out a specific commit. It's fine for looking around, but new commits there aren't on any branch, so to keep*

them I run `git switch -c new-branch`*. If I already switched away,* `git reflog` *lets me find the commit and recover it.*

Q31. Git/GitHub: how do you give a specific team member access to a repository, and grant them write permission?

Also asked as: How do you grant write permissions to a contributor in a GitHub repository?

In simple words: Hand out keys by role, not by person. Put people in teams (developers, QA, admins) and give each team the right level of key. When someone joins or leaves, you change the team, not every lock.

Detailed answer

- **Personal repository:** Settings -> Collaborators -> Add people. A collaborator on a personal repo gets write (push) access.
- **Organization repository (the right way):** add the person to a **team**, then give the team a role on the repo in Settings -> Collaborators and teams.
- **Roles:** Read (clone, open issues), Triage (manage issues and PRs), **Write** (push, merge PRs), Maintain (manage the repo without admin rights), Admin (settings, secrets, delete).
- **Write access still goes through branch protection:** protected `main`, required PR reviews, status checks and CODEOWNERS, so write doesn't mean "can push straight to production".
- Enforce SSO/2FA at the org level, review access regularly, and prefer fine-grained tokens or GitHub Apps for automation.

Example:

```
# Add a user as collaborator with write (push) permission
gh api -X PUT repos/my-org/meter-api/collaborators/jdoe -f permission=push

# Better: give a team write access
gh api -X PUT orgs/my-org/teams/backend-devs/repos/my-org/meter-api -f permission=push

# Check what someone has
gh api repos/my-org/meter-api/collaborators/jdoe/permission --jq .permission
```

What to say to the interviewer: *For an organization repo I add the person to the right team and give that team the Write role, instead of adding individuals, so onboarding and offboarding happen in one place. Write lets them push branches and merge PRs, but branch protection still requires reviews and passing checks on main. For a personal repo I'd add them as a collaborator, and in all cases I enforce 2FA or SSO and review access periodically.*

Q32. Git/GitHub: what is the difference between public and private repositories, and when would you use each?

In simple words: A public repo is a shop window: anyone on the street can look and copy the design. A private repo is the back office: only people you give a key to can enter.

Detailed answer

	Public	Private
Who can see it	Anyone on the internet	Only invited users or teams
Forking	Anyone	Only people with access (if allowed)
Typical use	Open source, samples, docs, portfolios	Company code, infrastructure, configs
Risk	Any leaked secret is exposed instantly and scraped by bots	Lower exposure, but secrets still don't belong in Git
GitHub Actions	Free minutes on hosted runners	Uses the plan's minute quota

- **Internal** repos (GitHub Enterprise) are visible to everyone in the company but not the public, which is good for inner-source.
- Company application code, Terraform, and anything with internal hostnames or architecture should be **private**.
- Before making a repo public, scan the whole history for secrets (gitleaks/trufflehog), not just the latest commit.

Example:

```
gh repo create my-org/infra-terraform --private
gh repo edit my-org/helm-charts --visibility public --accept-visibility-change-consequences
gitleaks detect --source . --log-opts="--all"     # scan the full history before going public
```

> ***What to say to the interviewer:*** *Public repos are visible to everyone and are for open-source work, samples or portfolios. Private repos are only for invited users or teams and are the default for company code and infrastructure. Before making anything public I scan the full Git history for secrets, because bots scrape public repos within minutes, and for company-wide sharing I use internal visibility.*

Q33. Git: how does squash work, when do you use it, and which commands do you run?

> **In simple words:** You wrote ten messy draft notes while working on one feature. Squashing staples them into one clean note, "Add meter-read retry logic", before handing it to the team.

Detailed answer

- **What it does:** combines several commits into one, so the main branch history shows one meaningful commit per feature or fix.
- **When to use it:** before merging a feature branch full of "wip", "fix typo" and "try again" commits, or when team policy says one commit per PR.
- **When not to:** when individual commits matter for review, bisect or reverts, and never on branches others have already pulled without coordinating (it rewrites history).

Three ways:

1. **Interactive rebase:** `git rebase -i HEAD~4`, then mark commits as `squash` or `fixup` and edit the message.

2. **Merge with squash:** `git merge --squash feature` stages all changes as one new commit.
3. **GitHub/GitLab "Squash and merge"** button on the PR, the most common in teams.

Example:

```
git log --oneline -4
# 9f1c2aa fix lint
# 7d3b1e0 wip tests
# 4a8e9c1 wip retry
# 1c2d3e4 add retry logic

git rebase -i HEAD~4
# pick 1c2d3e4 add retry logic
# squash 4a8e9c1 wip retry
# fixup 7d3b1e0 wip tests
# fixup 9f1c2aa fix lint

git push --force-with-lease        # only on your own feature branch

# alternative:
git checkout main && git merge --squash feature/retry && git commit -m "Add meter-read retry logic"
```

> ***What to say to the interviewer:*** *Squash combines several commits into one so the main branch has one clean commit per change. I do it with* `git rebase -i` *and squash or fixup, with* `git merge --squash`*, or most often with the Squash and merge button on the PR. Because it rewrites history, I only squash my own feature branches and push with* `--force-with-lease`*.*

Q34. Git: what is the difference between Git and GitHub?

> **In simple words:** Git is the camera that takes snapshots of your code. GitHub is the online photo album where you store the snapshots, share them, and let others comment on them.

Detailed answer

	Git	GitHub
What it is	A distributed version control tool	A cloud platform that hosts Git repositories
Runs where	On your machine, works offline	On the web (or GitHub Enterprise Server)
Made by	Linus Torvalds (2005), open source	GitHub Inc. (owned by Microsoft)
Does	Commits, branches, merges, history	PRs, code review, Issues, Actions CI/CD, Packages, security scanning, access control
Alternatives	Mercurial, SVN (centralized)	GitLab, Bitbucket, Azure Repos, Gitea

- You can use Git without GitHub, but GitHub is useless without Git.
- In DevOps, GitHub is often the **trigger point**: a push or PR fires webhooks or GitHub Actions.

Example:

```
git init && git add . && git commit -m "first commit"        # Git only, local
git remote add origin git@github.com:my-org/app.git          # now connect to GitHub
git push -u origin main
gh pr create --fill                                          # GitHub feature (PR)
```

What to say to the interviewer: *Git is the version control tool that tracks changes, branches and history, and it works fully offline on my laptop. GitHub is a hosting platform built around Git that adds pull requests, code review, Issues, Actions for CI/CD and access control. GitLab, Bitbucket and Azure Repos are alternatives to GitHub, not to Git.*

Q35. Git: what are the essential Git commands, and what does each do?

In simple words: Think of three places: your desk (working directory), a tray of things ready to file (the staging area), and the filing cabinet (commits), plus a shared office cabinet (the remote). Each command moves work between them.

Detailed answer

Command	What it does
`git init`	Creates a new repo (adds a hidden `.git` folder)
`git clone <url>`	Copies a remote repo locally with full history, linked to `origin`
`git status`	Shows changed, staged and untracked files
`git add <file>` / `git add .`	Stages changes for the **next commit**
`git commit -m "msg"`	Saves the staged snapshot to **local** history
`git push`	Uploads local commits to the remote branch (`-u origin <branch>` the first time)
`git fetch`	Downloads remote changes without touching your files
`git pull`	`fetch` + merge (or `--rebase`) into your current branch
`git branch` / `git switch -c x`	Lists branches / creates and switches to a new branch
`git checkout`	Older all-in-one command, now split into `switch` (branches) and `restore` (files)
`git merge` / `git rebase`	Integrates another branch
`git log --oneline --graph`	History
`git diff` / `git diff --staged`	Unstaged / staged changes
`git restore <file>`	Discards local changes to a file
`git reset` / `git revert`	Move a branch pointer (rewrites history) / create an undo commit (safe on shared branches)
`git tag v1.0.0`	Marks a release

Correcting common misconceptions:

- `git add` does not "compare with the remote"; it only stages changes for the next commit.
- `git push` does not create a repository; the remote repo must exist. It can create a new **branch** on the remote.

- `git checkout -b` creates a branch; the modern equivalent is `git switch -c`.

Example:

```
git clone git@github.com:my-org/meter-api.git && cd meter-api
git switch -c feature/retry
# ...edit files...
git status && git add src/retry.go && git commit -m "Add retry with backoff"
git push -u origin feature/retry
git fetch origin && git rebase origin/main      # stay current with main
```

> ***What to say to the interviewer:*** *The daily flow is clone, switch -c to create a branch, edit, add to stage, commit to save locally, and push to upload to the remote. I fetch or pull to get others' changes, and use merge or rebase to integrate. I also use status, log, diff and restore constantly, and I use revert rather than reset on shared branches because it doesn't rewrite history.*

Q36. Git: what is the difference between git push, git fetch and git pull?

> **In simple words:** fetch is checking the post box and bringing the letters inside without opening them. pull is bringing them in and immediately mixing them into your own papers. push is posting your own letters out.

Detailed answer

Command	Direction	Changes your working files?
`git fetch`	Remote -> local (updates `origin/*` only)	No, it's safe anytime
`git pull`	Remote -> local, then merge/rebase into the current branch	Yes, can cause conflicts
`git push`	Local -> remote	No (updates the remote branch)

- `git fetch` then `git log HEAD..origin/main` lets you **review** incoming changes before integrating them.
- `git pull --rebase` (or `git config pull.rebase true`) avoids noisy merge commits.
- `git push` is rejected if the remote has commits you don't have, so fetch/rebase first. Use `--force-with-lease`, never plain `--force`, on your own branches.

Example:

```
   remote (GitHub)      origin/main    your local main    working files
        |  <-- push --------------------------|
        |  -- fetch -->  updated              |              unchanged
        |  -- pull  -->  updated  --merge-->  updated        updated
```

```
git fetch origin
git log --oneline HEAD..origin/main      # what's new upstream?
git pull --rebase origin main
git push origin feature/retry
```

> ***What to say to the interviewer:*** `git fetch` *downloads new commits into the remote-tracking branches without touching my work, so it's always safe.* `git pull` *is fetch plus merge or rebase into my current branch, so it changes my files and can conflict.* `git push` *sends my commits to the remote and is rejected if the remote has moved ahead, so I fetch or rebase first.*

Q37. Git: how do you recover deleted changes (lost commits, a deleted branch, discarded files)?

In simple words: Git has a CCTV recording (the reflog) of everywhere your branches have pointed recently. Even if you "deleted" something, you can rewind the tape and take it back, as long as it was committed or stashed at some point.

Detailed answer

What was lost	How to recover
Commits after a bad `reset --hard` or rebase	`git reflog`, then `git reset --hard HEAD@{n}` or `git branch rescue <sha>`
A deleted branch	`git reflog` (or the SHA from `git branch -D` output), then `git branch <name> <sha>`
A dropped stash	`git fsck --lost-found`, look for dangling commits, then `git stash apply <sha>`
A file deleted but not committed	`git restore <file>` (from the last commit)
A file deleted in an old commit	`git log --diff-filter=D -- path` to find it, then `git checkout <sha>^ -- path`
A pushed change someone reverted	`git revert <revert-sha>` (revert the revert)
Uncommitted, never-staged edits	Git can't help; try the IDE local history or editor backups

- The reflog is local and entries expire (90 days by default), so recover sooner rather than later.
- Anything that was pushed also exists on the remote and in teammates' clones.

Example:

```
git reset --hard HEAD~3          # oops
git reflog
# e5f6a7b HEAD@{0}: reset: moving to HEAD~3
# 9c8b7a6 HEAD@{1}: commit: add billing export     <- what we want
git reset --hard 9c8b7a6

git branch -D feature/export     # oops: "Deleted branch feature/export (was 3d4e5f6)"
git branch feature/export 3d4e5f6
```

What to say to the interviewer: *For anything that was committed,* `git reflog` *is my safety net: it records where HEAD and the branches pointed, so I can reset back or create a branch from the lost SHA, including after a hard reset or a deleted branch. For dropped stashes I use* `git fsck --lost-found`*, for a deleted file* `git restore`*, and for a bad change on a shared branch I use* `git revert`*. Changes that were never committed or stashed can't be recovered by Git.*

Q38. Git: which useful Git commands are often ignored?

In simple words: Most people only use the steering wheel and pedals. These are the less-known dashboard buttons: rear camera, cruise control, lane assist. They save you when things go wrong.

Detailed answer

Command	Why it's useful
`git reflog`	Recover "lost" commits and branches
`git bisect start/good/bad`	Binary-search history to find the commit that introduced a bug
`git cherry-pick <sha>`	Copy one fix onto another branch (e.g. a hotfix onto a release branch)
`git blame -w -C <file>`	Who changed each line, ignoring whitespace and moved code
`git worktree add ../hotfix main`	Second working directory, so you can fix prod without stashing
`git stash -p` / `git add -p`	Stash or stage only selected hunks
`git log --oneline --graph --all`	Visual branch history
`git commit --fixup <sha>` + `git rebase -i --autosquash`	Clean fixes into the right commit
`git restore --staged <file>`	Unstage without losing changes
`git clean -nd`	Preview which untracked files would be deleted
`git shortlog -sn`	Commits per author
`git config rerere.enabled true`	Reuse recorded conflict resolutions during repeated rebases
`git switch -`	Jump back to the previous branch

Example:

```
git bisect start
git bisect bad HEAD
git bisect good v2.1.0
git bisect run ./scripts/test_meter_parse.sh      # finds the first bad commit automatically
git bisect reset

git worktree add ../app-hotfix release/2.3        # fix prod without disturbing current work
```

> ***What to say to the interviewer:*** *The ones I rely on that people often skip are reflog for recovery, bisect to find the commit that introduced a bug, cherry-pick to move a hotfix between branches, and worktree to work on a hotfix without stashing. I also use* `add -p` *for clean commits, fixup plus autosquash to tidy history, and* `clean -n` *to preview deletions.*

Advanced

Q39. Explain your CI/CD pipeline architecture, including approvals.

Also asked as: Describe the CI/CD workflow in your project. Explain your pipeline with CodePipeline/Jenkins and approvals.

> **In simple words:** Code travels on a conveyor belt through checkpoints: build, test, security scan, package, deploy to test rooms, and finally a manager signs off before it reaches customers.

Detailed answer

End-to-end flow (Jenkins + EKS example):

1. Developer opens a PR -> webhook triggers Jenkins multibranch job.
2. **CI stage:** checkout, build, unit tests, lint, SonarQube quality gate, dependency/secret scan.
3. On merge to main: Docker build, Trivy image scan, push to ECR with tag `version-gitsha`.
4. Deploy to **dev** automatically via Helm (or update the GitOps repo for ArgoCD).
5. Automated smoke/integration tests; promote the same image to **staging**.
6. **Manual approval** (Jenkins `input` with `submitter` restricted to release managers, or CodePipeline Manual Approval action with SNS email).
7. Deploy to **prod** with rolling/canary strategy; post-deploy health checks; auto-rollback on failure.
8. Notifications to Teams/Slack at each stage; metrics and logs watched in Grafana.

AWS-native equivalent: CodeCommit/GitHub -> CodePipeline -> CodeBuild (build/test/push ECR) -> Manual approval action -> CodeDeploy / EKS deploy.

Example:

```
Git PR --webhook--> Jenkins CI (build, test, Sonar, Trivy)
  --merge--> Docker build -> ECR (orders-api:1.4.0-a1b2c3d)
  --> Helm deploy DEV -> smoke tests
  --> Helm deploy STAGING -> integration tests
  --> [Manual Approval: release-managers]
  --> Helm deploy PROD (rolling / canary) -> health check -> auto rollback if failed
```

> ***What to say to the interviewer:*** *PRs trigger CI in Jenkins - build, unit tests, SonarQube and security scans. Merges build an immutable Docker image pushed to ECR, auto-deploy to dev via Helm, run smoke tests, promote to staging, and then a restricted manual approval gates production, where we deploy with a rolling or canary strategy and automatic rollback on failed health checks.*

Q40. Explain how a typical AWS-native CI/CD pipeline works end to end.

> **In simple words:** Think of an airport: check-in (source), security scan (build and test), boarding with a gate agent (approval), take-off in small groups (canary deploy), air traffic control watching the radar (monitoring), and the plane can turn back (rollback).

Detailed answer

1. **Source:** GitHub (via CodeStar/CodeConnections) or CodeCommit; a push or PR merge triggers CodePipeline through a webhook or EventBridge rule. S3 or ECR image push can also be a source.
2. **Build (CodeBuild or Jenkins):** `buildspec.yml` compiles, runs lint and unit tests, publishes test reports, builds the Docker image and pushes it to **ECR** (tag = commit SHA), or packages a zip to **S3**. Artifacts are versioned.
3. **Test and scan:** SonarQube quality gate, SCA/SAST, Trivy image scan, **Amazon Inspector** ECR scanning; integration tests in a test environment. Fail the pipeline on critical issues.
4. **Approval:** CodePipeline **Manual Approval** action (SNS email/Teams) before prod.
5. **Deploy: CodeDeploy** for EC2/ECS/Lambda (blue/green, canary `Canary10Percent5Minutes`, linear), **CloudFormation/CDK/Terraform** for infra,

Helm/kubectl/ArgoCD for **EKS**, **SAM** for Lambda.

6. **Rollback:** versioned immutable artifacts; CodeDeploy auto-rollback on failed deployment or **CloudWatch alarm**; ECS deployment circuit breaker; Lambda alias shifting back.
7. **Monitoring:** CloudWatch metrics/logs/alarms, SNS notifications, X-Ray traces, pipeline state change events via EventBridge to Teams/Slack.
8. **Security:** each stage has its own least-privilege IAM role; secrets from **Secrets Manager / Parameter Store**; cross-account deploys via assume-role into prod account; artifacts encrypted with KMS.

Improvement over a basic answer: mention **multi-account** (tools account deploys to dev/prod accounts), **build once promote everywhere**, and **alarm-based automatic rollback**.

Example:

```
# buildspec.yml
version: 0.2
phases:
  pre_build:
    commands:
      - aws ecr get-login-password | docker login -u AWS --password-stdin $ECR
      - TAG=${CODEBUILD_RESOLVED_SOURCE_VERSION:0:7}
  build:
    commands:
      - npm ci && npm run lint && npm test
      - docker build -t $ECR/orders:$TAG .
  post_build:
    commands:
      - docker push $ECR/orders:$TAG
      - printf '[{"name":"orders","imageUri":"%s"}]' $ECR/orders:$TAG > imagedefinitions.json
reports:
  unit: { files: ['reports/junit.xml'], file-format: JUNITXML }
artifacts:
  files: [imagedefinitions.json, appspec.yaml, taskdef.json]
```

> ***What to say to the interviewer:*** *A push to GitHub triggers CodePipeline through a webhook or EventBridge; CodeBuild runs lint and unit tests, builds the image and pushes it to ECR with the commit SHA. Scans like SonarQube, Trivy and Inspector gate it, a manual approval guards prod, and CodeDeploy does blue-green or canary on ECS, EC2 or Lambda with automatic rollback on CloudWatch alarms. Each stage has least-privilege IAM roles, secrets come from Secrets Manager, and we monitor with CloudWatch, SNS and X-Ray.*

Q41. What stages do you define in Jenkins, and how do you ensure full quality checks?

> **In simple words:** Quality checks are a series of filters - each one catches a different kind of dirt. Cheap and fast filters first, expensive ones later, and nothing reaches customers unless it passes all of them.

Detailed answer

Typical stages, fast to slow:

Stage	Tool examples	Gate
Checkout	git (shallow)	-

Stage	Tool examples	Gate
Lint / format	eslint, checkstyle, hadolint, tflint	fail on error
Unit tests	JUnit, pytest, jest	all pass
Coverage	JaCoCo, Istanbul	e.g. >= 80% on new code
SAST + quality	SonarQube / Semgrep	quality gate must pass
SCA (dependencies)	OWASP Dependency-Check, Snyk, Trivy fs	no CRITICAL
Secret scan	gitleaks	no findings
Build image	docker buildx	-
Image scan	Trivy, Inspector	no CRITICAL/HIGH fixable
Deploy dev + integration tests	Helm, Postman/newman, k6	pass
Approval	input step / environment	authorized approver
Deploy prod + smoke test	Helm --atomic	healthy, else rollback

How we enforce it:

- Gates **fail the build** (`abortPipeline: true`, `--exit-code 1`), not just report.
- Branch protection requires the checks before merge.
- The stages live in a **shared library**, so no team can quietly drop a check.
- Reports published (`junit`, coverage) so trends are visible.

Example:

```
stage('Quality') {
  parallel {
    stage('Lint')    { steps { sh 'npm run lint' } }
    stage('Unit')    { steps { sh 'npm test -- --coverage' } }
    stage('Secrets') { steps { sh 'gitleaks detect --exit-code 1' } }
    stage('SCA')     { steps { sh 'trivy fs --severity CRITICAL --exit-code 1 .' } }
  }
}
stage('Sonar Gate') {
  steps {
    withSonarQubeEnv('sonar') { sh 'sonar-scanner' }
    timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
  }
}
```

> ***What to say to the interviewer:*** *Our stages go from fast to slow: lint, unit tests with a coverage gate, SonarQube quality gate, dependency and secret scanning, image build and Trivy scan, deploy to dev with integration tests, approval, and prod deploy with smoke tests. Every gate fails the build rather than just reporting, branch protection requires them before merge, and they live in a shared library so they cannot be skipped.*

Q42. Write a CI/CD Jenkins pipeline that you have implemented in your project.

Also asked as: Walk me through a Jenkins pipeline you have written.

In simple words: This is the full recipe card - every step the code goes through from GitHub to production on EKS.

Detailed answer

Key design points: runs on Kubernetes pod agents, AWS access via IAM role (no static keys), immutable image tags, quality gate and image scan before push, auto-deploy to dev, approval gate for prod, rollback on failure.

Example:

```
pipeline {
  agent { label 'k8s-docker' }
  options {
    timeout(time: 45, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
    disableConcurrentBuilds()
    timestamps()
  }
  environment {
    AWS_REGION = 'ap-south-1'
    ECR        = '123456789012.dkr.ecr.ap-south-1.amazonaws.com'
    APP        = 'orders-api'
    TAG        = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build & Unit Test') {
      steps { sh 'mvn -B clean verify' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Code Quality') {
      steps {
        withSonarQubeEnv('sonar') { sh 'mvn -B sonar:sonar' }
        timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
      }
    }
    stage('Docker Build') { steps { sh 'docker build -t $ECR/$APP:$TAG .' } }
    stage('Image Scan') {
      steps { sh 'trivy image --exit-code 1 --severity CRITICAL,HIGH $ECR/$APP:$TAG' }
    }
    stage('Push to ECR') {
      when { branch 'main' }
      steps {
        sh '''
          aws ecr get-login-password --region $AWS_REGION |
            docker login --username AWS --password-stdin $ECR
          docker push $ECR/$APP:$TAG
        '''
      }
    }
    stage('Deploy DEV') {
      when { branch 'main' }
      steps {
        sh '''
          aws eks update-kubeconfig --name dev-eks --region $AWS_REGION
          helm upgrade --install $APP charts/$APP -n dev \
            --set image.tag=$TAG --wait --timeout 5m --atomic
        '''
      }
    }
```

```
    stage('Approve PROD') {
      when { branch 'main' }
      steps {
        timeout(time: 2, unit: 'HOURS') {
          input message: "Deploy $TAG to PROD?", submitter: 'release-managers'
        }
      }
    }
    stage('Deploy PROD') {
      when { branch 'main' }
      steps {
        sh '''
          aws eks update-kubeconfig --name prod-eks --region $AWS_REGION
          helm upgrade --install $APP charts/$APP -n prod \
            --set image.tag=$TAG --wait --timeout 10m --atomic
        '''
      }
    }
  }
  post {
    success { echo "Deployed ${env.TAG}" }
    failure { echo 'Send Teams/Slack failure alert with build URL' }
    always  { sh 'docker rmi $ECR/$APP:$TAG || true'; cleanWs() }
  }
}
```

What to say to the interviewer: *Our Jenkinsfile runs on Kubernetes agents with an IAM role for AWS. It builds and unit-tests, enforces a SonarQube quality gate, builds a Docker image tagged with build number and commit SHA, scans it with Trivy, pushes to ECR, and deploys to dev with helm upgrade --atomic. Production needs an approval from release managers and uses the same image tag, with atomic Helm rollback if health checks fail.*

Q43. What kinds of applications do you deploy through Jenkins, and with which deployment tools?

In simple words: Different parcels need different delivery vans - a container goes by Helm truck to Kubernetes, a serverless function goes by SAM courier to Lambda, a server config goes by Ansible.

Detailed answer

App type	Artifact	Deploy tool
Containerized microservices (Java/Node/Go APIs)	Docker image in ECR	Helm / kubectl to EKS, or ArgoCD (GitOps)
ECS services	Docker image + task definition	CodeDeploy blue/green, `aws ecs update-service`
Serverless functions	zip in S3	SAM / Serverless / Terraform to Lambda
Static frontends (React/Angular)	build folder	`aws s3 sync` + CloudFront invalidation
VM-based apps	jar/package	Ansible, CodeDeploy agent
Infrastructure	Terraform code / plan	`terraform plan` + approval + `apply`
Config / monitoring	dashboards, alert rules	Grafana API, Helm (kube-prometheus-stack)

Why this matters: same pipeline template, different final deploy stage. Kubernetes apps prefer Helm with `--atomic` or ArgoCD because they give rollback history and drift correction.

Example:

```
stage('Deploy Frontend') {
  steps {
    sh 'aws s3 sync build/ s3://web-prod-bucket --delete'
    sh 'aws cloudfront create-invalidation --distribution-id $CF_ID --paths "/*"'
  }
}
stage('Deploy Lambda') { steps { sh 'sam deploy --stack-name reports --no-confirm-changeset' } }
stage('Deploy EKS')    { steps { sh 'helm upgrade --install api charts/api --set image.tag=$TAG --at
    omic' } }
```

> ***What to say to the interviewer:*** *Mostly containerized microservices deployed to EKS with Helm or through ArgoCD, plus Lambda functions via SAM or Terraform, static frontends to S3 with CloudFront invalidation, and Terraform infrastructure with plan and approval. The pipeline template is shared, and only the deploy stage changes per application type.*

Q44. How do you handle the CD aspect and promote builds across environments (Dev -> Staging -> Prod)?

Also asked as: How do you handle continuous delivery in your projects? Explain multi-environment promotion flows. How do you manage dev, staging and prod environments in CI/CD?

> **In simple words:** You bake one cake, taste it in the test kitchen, then in the dress-rehearsal kitchen, and only then serve the SAME cake to guests - you never bake a new one for the guests.

Detailed answer

Principles:

- **Build once, promote the same artifact:** the image tag tested in dev/staging is exactly what goes to prod. Never rebuild for prod.

- **Environment config is separate** from the artifact: Helm values per env (`values-dev.yaml`, `values-prod.yaml`), ConfigMaps, secrets from Secrets Manager.
- **Gates between environments:** automated tests (smoke, integration, performance), quality gates, and a manual approval for prod.
- **Progressive rollout in prod:** rolling/canary with health checks and automatic rollback.
- **GitOps option:** promotion = a PR that updates the image tag in the env folder of a config repo; ArgoCD syncs it. Git history becomes the audit log.
- **Isolate the environments themselves:** separate namespaces or clusters, ideally separate cloud accounts/subscriptions for prod; per-env deploy identities and secrets, so a dev pipeline physically cannot touch prod.
- **Environment objects in the CI tool:** GitHub/GitLab environments or Azure DevOps Environments carry protection rules (approvals, wait timers, allowed branches) and env-scoped secrets; Jenkins uses folder-scoped credentials plus an `input` gate.
- **Keep environments alike:** the same Terraform modules and Helm chart for every env, only sizes and endpoints differ, so "works in staging" means something in prod.

Example:

```
gitops-repo/
  apps/orders-api/
    base/
    envs/dev/values.yaml       image.tag: 1.4.0-a1b2c3d
    envs/staging/values.yaml   image.tag: 1.4.0-a1b2c3d
    envs/prod/values.yaml      image.tag: 1.3.2-9f8e7d6   <- promote via PR

# Promote to prod = PR that changes one line, approved by release manager
yq -i '.image.tag = "1.4.0-a1b2c3d"' apps/orders-api/envs/prod/values.yaml
```

> ***What to say to the interviewer:*** *We build the artifact once and promote the same immutable image tag through dev, staging and prod, with only environment config differing through per-env Helm values. Each promotion is gated by automated tests, and prod needs an approval. With GitOps, promotion is simply a reviewed pull request that bumps the image tag in the prod folder, and ArgoCD applies it.*

Q45. How do you connect Jenkins to AWS securely and push Docker images for multiple services to ECR?

Also asked as: How do you connect multiple services in a Jenkins pipeline and push images to AWS ECR securely using secrets?

> **In simple words:** Instead of giving Jenkins a permanent key to AWS, give it a temporary visitor badge that expires by itself.

Detailed answer

Secure options, best first:

1. **Jenkins on EC2/EKS -> IAM role:** EC2 instance profile or EKS IRSA/Pod Identity for the agent pod. No keys stored anywhere; credentials rotate automatically.
2. **Cross-account:** agent role does `sts:AssumeRole` into the target account's deploy role.

3. **Jenkins outside AWS:** OIDC federation or, last resort, AWS credentials in Jenkins Credentials (AWS Credentials plugin) with least-privilege policy and regular rotation.

- The IAM policy should allow only `ecr:GetAuthorizationToken` plus push actions on specific repositories.

For many services: loop or `parallel` over a list of services (monorepo) or a shared library function `buildAndPush(service)`; only build services whose folder changed.

Example:

```
def services = ['orders', 'payments', 'users']
pipeline {
  agent { label 'docker' }          // agent has IAM role via instance profile / IRSA
  environment { ECR = '123456789012.dkr.ecr.ap-south-1.amazonaws.com'; TAG = "${GIT_COMMIT.take(7)}"
    }
  stages {
    stage('ECR Login') {
      steps { sh 'aws ecr get-login-password | docker login -u AWS --password-stdin $ECR' }
    }
    stage('Build & Push') {
      steps {
        script {
          def jobs = [:]
          services.each { s ->
            jobs[s] = {
              if (sh(script: "git diff --quiet HEAD~1 -- services/${s}", returnStatus: true) != 0) {
                sh "docker build -t $ECR/${s}:$TAG services/${s} && docker push $ECR/${s}:$TAG"
              }
            }
          }
          parallel jobs
        }
      }
    }
  }
}
```

What to say to the interviewer: *Jenkins agents run on AWS with an IAM role - instance profile or IRSA - so there are no static keys, and cross-account deploys use sts assume-role. The role only has ECR push rights on specific repos. For multiple services we build and push in parallel, only for services whose folder changed, each tagged with the commit SHA.*

Q46. How do Jenkins and a Kubernetes cluster communicate?

In simple words: Jenkins needs a pass (kubeconfig or cloud identity) to talk to the Kubernetes front desk (API server). With GitOps, Jenkins does not even enter - it just leaves a note in Git and ArgoCD inside the cluster picks it up.

Detailed answer

Two directions:

- **Jenkins uses Kubernetes for agents:** Kubernetes plugin creates build pods on demand. Configured under Manage Jenkins -> Clouds with the API server URL and a service account / credential.
- **Jenkins deploys to Kubernetes:**

- Agent has `kubectl`/`helm` and a kubeconfig. On EKS: `aws eks update-kubeconfig` using the agent's IAM role, which is mapped to Kubernetes RBAC via EKS access entries (or aws-auth ConfigMap).
- The deploy identity should have a namespace-scoped Role, not cluster-admin.
- Store kubeconfig (if used) as a Secret file credential.
- **GitOps (pull) model:** Jenkins only updates the manifest repo; ArgoCD in the cluster pulls and applies. Cluster credentials never leave the cluster - most secure.

Example:

```
# EKS: map Jenkins IAM role to a k8s group (access entries)
aws eks create-access-entry --cluster-name prod-eks \
  --principal-arn arn:aws:iam::123456789012:role/jenkins-deployer
aws eks associate-access-policy --cluster-name prod-eks \
  --principal-arn arn:aws:iam::123456789012:role/jenkins-deployer \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSEditPolicy \
  --access-scope type=namespace,namespaces=orders

# In pipeline
withCredentials([file(credentialsId: 'kubeconfig-dev', variable: 'KUBECONFIG')]) {
  sh 'kubectl -n orders rollout status deploy/orders-api'
}
```

> ***What to say to the interviewer:*** *Jenkins talks to the Kubernetes API server either through a kubeconfig credential or, on EKS, through its IAM role mapped to namespace-scoped RBAC using access entries. We also use the Kubernetes plugin so builds run in ephemeral pods. For production we prefer GitOps, where Jenkins only updates the manifest repo and ArgoCD inside the cluster pulls the change.*

Q47. How can Jenkins be integrated with other DevOps tools?

> **In simple words:** Jenkins is the conductor of an orchestra - it does not play every instrument, it tells Git, Maven, SonarQube, Docker, Trivy, Helm and Slack when to play.

Detailed answer

Integrations through plugins, CLIs and webhooks:

Area	Tools	How
Source control	GitHub, GitLab, Bitbucket	Webhooks, Git/Branch Source plugins
Build	Maven, Gradle, npm	Tools config, container agents
Code quality	SonarQube	`withSonarQubeEnv`, `waitForQualityGate`
Security	Trivy, Snyk, OWASP Dependency-Check, gitleaks	CLI steps, fail on severity
Artifacts	Nexus, Artifactory, ECR, S3	Plugins or CLI push
Containers	Docker, Kubernetes	Docker Pipeline plugin, Kubernetes plugin
Deploy	Helm, kubectl, ArgoCD, Terraform, Ansible	CLI in stages

Area	Tools	How
Secrets	Vault, AWS Secrets Manager	Vault plugin, IAM roles
Notify	Teams, Slack, email, Jira	Notification plugins, Jira plugin
Monitoring	Prometheus	Prometheus metrics plugin -> Grafana

Example:

```
stage('Terraform Plan') { steps { sh 'terraform init && terraform plan -out=tfplan' } }
stage('Ansible')        { steps { sh 'ansible-playbook -i inventory/dev site.yml' } }
stage('Jira update')    { steps { jiraComment issueKey: 'PC-123', body: "Deployed ${TAG}" } }
```

> ***What to say to the interviewer:*** *Jenkins integrates through plugins, CLIs and webhooks - GitHub webhooks to trigger, Maven or npm to build, SonarQube and Trivy for quality and security, ECR or Nexus for artifacts, Helm, Terraform and Ansible to deploy, Vault or Secrets Manager for secrets, and Teams or Slack plus Jira for notifications. We also export Jenkins metrics to Prometheus and Grafana.*

Q48. What are Jenkins Shared Libraries, and how do you use them to reduce or modularize a large Jenkinsfile?

Also asked as: How can you reduce the size of a Jenkinsfile? How do you refactor a Jenkinsfile that exceeds practical limits? How are shared libraries written, defined and integrated into Jenkinsfiles?

> **In simple words:** Instead of every team writing the same 500-line recipe, put the common recipe in a shared cookbook and each team just writes "use the standard recipe for a Java app".

Detailed answer

Why it matters: huge Jenkinsfiles are hard to maintain and can hit the JVM "Method code too large" (64KB) limit.

A **Shared Library** is a separate Git repo of reusable Groovy pipeline code. Structure:

- `vars/` - global steps; `vars/helmDeploy.groovy` with a `call()` method becomes a step `helmDeploy(...)`.
- `src/` - Groovy classes in packages (e.g. `org/company/Docker.groovy`) for complex logic.
- `resources/` - non-Groovy files (templates, scripts) loaded with `libraryResource`.

Defining: Manage Jenkins -> System -> **Global Pipeline Libraries** -> name `ci-lib`, default version (branch/tag), Git repo URL, credentials. Global libraries are trusted (run outside the sandbox), so only admins should change them; folder-level libraries are sandboxed.

Integrating: `@Library('ci-lib@v2') _` at the top of the Jenkinsfile (pin a tag for stability), or `library 'ci-lib@v2'` dynamically. Version the library with tags and test changes on a branch before moving the tag.

Techniques to shrink the Jenkinsfile:

- **Shared Libraries:** move common steps into the library. Each app Jenkinsfile becomes 5-10 lines.
- **Move logic into scripts:** long shell blocks go into `scripts/build.sh`, called from the pipeline.

- **Parameterize and loop** instead of copy-pasted stages; use `matrix` for OS/version combinations.
- **Templates:** Jenkins Templating Engine or a standard pipeline per app type (java, node, helm).
- **Split pipelines:** separate CI and CD jobs, or trigger downstream jobs.
- Keep heavy Groovy logic out of the controller; avoid giant `script {}` blocks.

Example:

```
// ci-lib repo: vars/javaServicePipeline.groovy
def call(Map cfg) {
  pipeline {
    agent { label 'docker' }
    stages {
      stage('Build')  { steps { sh 'mvn -B clean verify' } }
      stage('Image')  { steps { buildAndPushImage(cfg.app) } }
      stage('Deploy') { when { branch 'main' } steps { helmDeploy(cfg.app, 'dev') } }
    }
  }
}

// ci-lib repo: vars/helmDeploy.groovy  -> usable as step helmDeploy('orders-api', 'dev')
def call(String app, String env) {
  def values = libraryResource "helm/values-${env}.yaml"   // from resources/
  writeFile file: 'values.yaml', text: values
  sh "helm upgrade --install ${app} charts/${app} -n ${env} -f values.yaml --atomic"
}

// Application Jenkinsfile - now only 3 lines
@Library('ci-lib@v2') _
javaServicePipeline(app: 'orders-api')
```

> ***What to say to the interviewer:*** *A shared library is a separate Git repo with vars for global steps, src for Groovy classes and resources for templates, registered under Global Pipeline Libraries and loaded with @Library pinned to a version tag. I move common logic there so each app Jenkinsfile becomes a few lines calling a standard pipeline, move long shell logic into scripts, and replace duplicated stages with loops or matrix - this avoids the method-too-large limit and lets us fix the pipeline once for all services.*

Q49. What rollback strategies do you use, and how do you version artifacts?

Also asked as: How would you make a deployment rollback-safe? Your actual rollout + rollback strategy, not buzzwords.

> **In simple words:** If every box leaving the factory has a unique label, going back to "yesterday's good box" is easy. If every box is just called "latest", you cannot tell which one was good.

Detailed answer

Artifact versioning:

- Semantic version + git SHA: `1.4.0-a1b2c3d`, or build number + SHA. Never deploy `latest` to prod.
- Enable **ECR tag immutability** so a tag cannot be overwritten.
- Record which version is in which environment (GitOps repo, Helm release history, deployment dashboard).

Rollback strategies:

- **Redeploy previous version:** `helm rollback`, `kubectl rollout undo`, or revert the GitOps commit.
- **Blue-Green:** switch traffic back to the old environment.
- **Canary:** set new version weight to 0.
- **Feature flags:** turn the feature off without redeploying.
- **Automatic:** `helm --atomic`, Argo Rollouts analysis, CodeDeploy auto-rollback on CloudWatch alarm.
- **Database:** use backward-compatible (expand/contract) migrations so old code still works with the new schema - otherwise app rollback is not safe.

Rollback-safe checklist (the actual rollout + rollback strategy, not buzzwords):

1. **Immutable, versioned artifacts:** deploy by image **digest** (`@sha256:...`) or an immutable tag; the same artifact is promoted through all environments.
2. **Keep previous versions ready:** `revisionHistoryLimit` keeps the last N ReplicaSets; Helm keeps release history (`--history-max`); old images are not deleted by ECR lifecycle rules too early.
3. **Backward-compatible DB migrations (expand/contract):** the previous app version must still work against the new schema.
4. **Feature flags:** risky behaviour can be switched off without a redeploy.
5. **Config versioned together with code:** Helm values, ConfigMaps and flags live in Git with the release, so a rollback restores the matching config too.
6. **Progressive rollout with automated analysis and abort:** canary steps checked against error rate and latency (Argo Rollouts, CodeDeploy alarms), aborting automatically.
7. **One-command rollback, practised regularly:** `helm rollback`, `kubectl rollout undo`, or `git revert` in GitOps - rehearsed in game days, and documented in the runbook.
8. **No manual hotfixes that skip Git:** a `kubectl edit` in prod is lost on the next sync or rollback; every fix goes through the pipeline.

Example:

```
spec:
  revisionHistoryLimit: 10                 # keep last 10 ReplicaSets for rollout undo
  template:
    spec:
      containers:
      - name: orders-api
        image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-api@sha256:4f1c...

helm upgrade --install orders-api charts/orders-api --history-max 10 --atomic
helm history orders-api -n prod
helm rollback orders-api 41 -n prod --wait

kubectl rollout history deploy/orders-api -n prod
kubectl rollout undo deploy/orders-api -n prod --to-revision=7

aws ecr put-image-tag-mutability --repository-name orders-api \
  --image-tag-mutability IMMUTABLE
```

> ***What to say to the interviewer:*** *Every artifact gets an immutable version tag with the git SHA, with ECR tag immutability on, so we always know exactly what is running. Rollback is a Helm rollback or*

a git revert in the GitOps repo, Blue-Green switch-back or canary weight to zero, and we use feature flags and backward-compatible database migrations so rolling back the app is always safe. To make it truly rollback-safe we deploy by digest, keep the last N releases, version config with code, use canary with automatic abort, practise the one-command rollback regularly, and never hotfix production outside Git.

Q50. What are secure pipeline (DevSecOps) practices?

In simple words: The pipeline has the keys to production, so it must be guarded like the vault of a bank - checks at every door, no spare keys lying around, and cameras recording everything.

Detailed answer

- **Shift-left scanning:** SAST (SonarQube/Semgrep), dependency/SCA (OWASP Dependency-Check, Snyk), secret scanning (gitleaks), IaC scanning (Checkov/tfsec), container scanning (Trivy). Fail the build on critical findings.
- **No static secrets:** IAM roles/OIDC for cloud, Vault/Secrets Manager at runtime, masked credentials.
- **Least privilege:** deploy role scoped to one account/namespace; separate roles for dev and prod.
- **Protect the pipeline itself:** branch protection, required reviews, signed commits; pipeline definition changes reviewed; restrict who can approve prod.
- **Ephemeral isolated agents:** fresh pod per build; no untrusted PR code on agents that hold prod credentials.
- **Supply chain:** pin base image digests and action versions (by SHA), generate SBOM, sign images (cosign) and verify at deploy.
- **Audit:** keep build logs, who approved what, and deploy history.

Example:

```
stage('Security Scans') {
  parallel {
    stage('Secrets') { steps { sh 'gitleaks detect --source . --exit-code 1' } }
    stage('Deps')    { steps { sh 'trivy fs --exit-code 1 --severity CRITICAL .' } }
    stage('IaC')     { steps { sh 'checkov -d infra/ --quiet' } }
  }
}
stage('Sign image') { steps { sh 'cosign sign --key awskms:///alias/cosign $IMAGE@$DIGEST' } }
```

What to say to the interviewer: *I build security into every stage: secret, dependency, SAST, IaC and image scanning with builds failing on critical issues. Pipelines use short-lived IAM or OIDC credentials instead of stored keys, run on ephemeral agents with least-privilege deploy roles, have protected branches and restricted prod approvals, and images are pinned, scanned and signed before deploy.*

Q51. How do you secure secrets across CI/CD pipelines?

Also asked as: You are asked to secure secrets across CI/CD pipelines. What is your preferred approach?

In simple words: The best secret is one that does not exist - use temporary badges instead of permanent keys. For the secrets you must have, keep them in one central safe and let the pipeline

borrow them only for a moment.

Detailed answer

My preferred approach, in order:

1. **Eliminate long-lived cloud keys:** GitHub Actions/GitLab/Azure DevOps -> OIDC federation to AWS/Azure/GCP; Jenkins on AWS -> IAM roles/IRSA.
2. **Central secrets manager:** HashiCorp Vault or AWS Secrets Manager / Azure Key Vault; pipelines fetch at runtime with a short-lived identity.
3. **CI-native secret store** only for what remains (Jenkins Credentials, GitHub encrypted secrets, Azure DevOps variable groups linked to Key Vault), scoped per environment with approvals on prod environments.
4. **Hygiene:** mask in logs, never echo, never bake into Docker images or artifacts, `no_log` in Ansible.
5. **Detect leaks:** gitleaks pre-commit + pipeline scan, GitHub secret scanning.
6. **Rotate** regularly and immediately on any exposure.

Example:

```
# GitHub Actions -> AWS with OIDC (no stored AWS keys)
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production          # requires reviewers
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
          aws-region: ap-south-1
      - run: |
          DB_PASS=$(aws secretsmanager get-secret-value --secret-id prod/db \
            --query SecretString --output text)
          echo "::add-mask::$DB_PASS"
```

What to say to the interviewer: *My first step is removing static keys by using OIDC federation or IAM roles, so pipelines get short-lived credentials. Remaining secrets live in a central manager like Vault or Secrets Manager and are fetched at runtime, scoped per environment, masked in logs, never baked into images, and leak scanning plus regular rotation cover the rest.*

Q52. How do you ensure secure and dynamic secret rotation in Azure DevOps pipelines?

In simple words: Instead of copying the password into the pipeline (and forgetting to update it), point the pipeline at the safe (Key Vault). When the password in the safe changes, the pipeline automatically uses the new one.

Detailed answer

- Store secrets in **Azure Key Vault**, not as plain pipeline variables.

- Link a **Variable Group to Key Vault** or use the `AzureKeyVault@2` task - secrets are fetched at each run, so rotation in Key Vault needs no pipeline change.
- Connect with a **Workload Identity Federation service connection** (OIDC) - no client secret to rotate for the pipeline itself.
- Enable **Key Vault rotation policies** (for keys) or Event Grid `SecretNearExpiry` -> Azure Function to rotate app secrets (e.g. regenerate storage keys, update SQL password).
- Apps read secrets at runtime (Managed Identity, Key Vault references, CSI Secret Store driver on AKS) so they pick up new versions.
- Protect with RBAC, environment approvals and checks, and audit via Key Vault diagnostic logs.

Example:

```
variables:
- group: prod-kv-secrets          # variable group linked to Key Vault

steps:
- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'sc-prod-oidc'   # workload identity federation
    KeyVaultName: 'kv-payments-prod'
    SecretsFilter: 'DbPassword,ApiKey'
    RunAsPreJob: true
- script: ./deploy.sh
  env:
    DB_PASSWORD: $(DbPassword)          # masked in logs
```

> ***What to say to the interviewer:*** *I keep secrets in Azure Key Vault and fetch them at runtime via a Key Vault-linked variable group or the AzureKeyVault task, using a workload identity federation service connection so the pipeline itself has no secret. Rotation happens in Key Vault through rotation policies or an Event Grid-triggered function, and because pipelines and apps read the latest version at runtime, no pipeline changes are needed.*

Q53. Azure DevOps: How do you secure access to artifacts and feeds (Azure Artifacts)?

> **In simple words:** A feed is a company warehouse of packages. You decide who can only take things out (Reader), who can put things in (Contributor), and who manages the warehouse (Owner). Outside goods only come in through one checked gate (upstream sources).

Detailed answer

- **Feed permissions (least privilege):** Reader (download), Collaborator (download + save packages from upstream), Contributor (publish), Owner (manage feed and permissions). Developers usually get Reader; only CI publishes.
- **Project-scoped vs organization-scoped feeds:** project-scoped feeds inherit the project's visibility and permissions and are the recommended default; org-scoped feeds are visible across the organization, so use them only for truly shared packages.
- **Pipeline identity:** pipelines authenticate as the **Project Build Service** (or Project Collection Build Service) identity - grant it Contributor only on the feeds it publishes to, and Reader elsewhere. Use `NuGetAuthenticate`/`npmAuthenticate` tasks instead of stored tokens.

Limit job authorization scope to the current project.

- **Upstream sources:** proxy npmjs, PyPI, NuGet or Maven Central through the feed so every external package is cached and auditable, and not pulled directly by developers; this also reduces dependency-confusion risk because internal package names resolve from your feed first.

- **Views:** `@local`, `@prerelease`, `@release` - promote a package to `@release` only after it passes testing; consumers and prod builds only read the `@release` view.

- **PATs:** if a PAT is needed (e.g. a developer laptop), scope it to Packaging (Read) only with a short expiry; admins can restrict full-scope PATs by policy.

- **Service connections to external systems:** use **workload identity federation** (OIDC) instead of secrets, with pipeline permissions and approvals on the connection.

- **Retention policies:** keep the last N versions per package and delete old ones (packages promoted to views are kept) - controls cost and removes stale vulnerable versions.

- **Audit:** Azure DevOps auditing streamed to Log Analytics.

Example:

```
steps:
- task: npmAuthenticate@0              # uses the build service identity, no PAT
  inputs: { workingFile: .npmrc }
- script: npm ci && npm publish

# .npmrc
registry=https://pkgs.dev.azure.com/org/project/_packaging/internal-feed/npm/registry/
always-auth=true

# Consumers of tested packages read only the release view
registry=https://pkgs.dev.azure.com/org/project/_packaging/internal-feed@release/npm/registry/
```

> ***What to say to the interviewer:*** *I use project-scoped feeds with least-privilege roles - developers read, only the pipeline's build service identity can publish - and pipelines authenticate with the npm or NuGet authenticate tasks rather than PATs. External packages come only through upstream sources, tested versions are promoted to the release view that prod builds consume, any PATs are packaging-read-only and short-lived, service connections use workload identity federation, and retention policies and audit logs keep it clean.*

Q54. How do you manage deployments and rollbacks for 100+ microservices on EKS/ECS?

> **In simple words:** With 100 services you cannot hand-deploy each one. You write the desired state of every service in one Git notebook, and a robot (ArgoCD) keeps the cluster matching that notebook - rolling back is just undoing a page in the notebook.

Detailed answer

- **Standardize:** one shared pipeline template / Jenkins shared library and one base Helm chart for all services; each service only provides values.

- **GitOps with ArgoCD:** a config repo holds manifests per service per environment. ArgoCD `ApplicationSet` generates Applications for all services automatically. CI only builds images and bumps tags.

- **Independent deploys:** each service deploys on its own; no big-bang releases.
- **Progressive delivery:** Argo Rollouts canary with automated analysis on Prometheus metrics, auto-abort on errors. On ECS: CodeDeploy blue/green with CloudWatch alarm rollback, or ECS deployment circuit breaker.
- **Rollback:** `git revert` the tag bump -> ArgoCD syncs the old version; or `argo rollouts abort/undo`.
- **Visibility:** dashboard of versions per environment, deploy events annotated on Grafana, alerting per service.
- **Guardrails:** sync windows, PodDisruptionBudgets, per-service SLOs.

Example:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: prod-services }
spec:
  generators:
  - git:
      repoURL: https://github.com/org/gitops.git
      revision: main
      directories: [ { path: "apps/*/envs/prod" } ]
  template:
    metadata: { name: '{{path[1]}}-prod' }
    spec:
      project: prod
      source: { repoURL: https://github.com/org/gitops.git, targetRevision: main,
                path: '{{path}}' }
      destination: { server: https://kubernetes.default.svc, namespace: '{{path[1]}}' }
      syncPolicy: { automated: { prune: true, selfHeal: true } }

# Rollback one service
git revert <commit-that-bumped-orders-tag> && git push
```

What to say to the interviewer: *At that scale I standardize on one pipeline template and one base Helm chart, and use GitOps with ArgoCD ApplicationSets so every service's desired version lives in Git. CI builds and bumps the tag, Argo Rollouts does canary with Prometheus-based automatic abort, and rollback is a git revert that ArgoCD syncs - on ECS the equivalent is CodeDeploy blue/green or the deployment circuit breaker.*

Q55. How do you ensure zero-downtime deployments? Share a real example.

In simple words: Like changing the tyres of a bus while it keeps driving - you only remove an old tyre after the new one is firmly on and holding weight.

Detailed answer

Strategy-level ingredients:

- **Deployment strategy:** rolling (with maxUnavailable 0), Blue-Green, or Canary.
- **Health checks:** readiness probes / target group health checks so traffic only goes to ready instances.
- **Graceful shutdown:** connection draining (deregistration delay, `preStop` sleep, handle SIGTERM).
- **Enough capacity:** min replicas >= 2, PodDisruptionBudget, surge capacity.

- **Backward-compatible changes:** API versioning and expand/contract DB migrations, so old and new versions can run side by side.
- **Automated verification and rollback:** smoke tests, metrics-based canary analysis, `helm --atomic`.

Real example (sample - adapt to your project): an API on EKS behind an ALB. We set `maxUnavailable: 0, maxSurge: 25%`, readiness probe on `/health`, `preStop` sleep 15s, and ALB deregistration delay 30s. A DB column rename was done in 3 releases (add new column, dual-write, drop old). The pipeline ran `helm upgrade --atomic`; one release failed readiness and Helm rolled back automatically with zero user errors.

Example:

```
strategy:
  type: RollingUpdate
  rollingUpdate: { maxUnavailable: 0, maxSurge: 25% }
...
lifecycle:
  preStop:
    exec: { command: ["sh", "-c", "sleep 15"] }
readinessProbe:
  httpGet: { path: /health, port: 8080 }

helm upgrade --install api charts/api --set image.tag=$TAG --atomic --wait --timeout 10m
```

> ***What to say to the interviewer:*** *Zero downtime needs a rolling, blue-green or canary strategy, readiness checks so only healthy instances get traffic, graceful connection draining, enough replicas, and backward-compatible database changes. In practice we deploy with maxUnavailable zero, readiness probes, a preStop delay and helm atomic, so a bad release fails its checks and rolls back automatically without users noticing.*

Q56. How would you optimize a slow CI/CD pipeline (e.g. 40 minutes for a small change)?

Also asked as: How would you optimize a CI/CD pipeline? Pipeline optimization for 70%+ faster deployments. How does caching work in pipelines? / CI builds 40 Docker images and takes 18 minutes. How do you optimize it?

> **In simple words:** If a car wash takes 40 minutes, time each station first. Then stop re-washing clean parts (caching), run stations side by side (parallel), and skip stations that do not apply to this car (change detection).

Detailed answer

Step-by-step:

1. **Measure first:** stage timing view / Blue Ocean / GitHub Actions timing - find the top 2-3 slow stages.
2. **Cache:** dependency caches (`~/.m2`, `node_modules`, pip), Docker layer cache (BuildKit `--cache-from`, registry cache), Terraform plugin cache.
3. **Parallelize:** unit tests, lint, scans in `parallel`; split test suites across agents (test sharding).
4. **Build only what changed:** in monorepos run only affected services (path filters, `git diff`).
5. **Optimize Dockerfile:** copy dependency files first, multi-stage, slim base images, `.dockerignore`.

6. **Right test at right stage:** fast unit tests on PR; heavy e2e/perf tests nightly or only before prod.
7. **Faster agents:** warm pools of agents, bigger instances for build stage, avoid queue waiting; ephemeral pods with pre-pulled images.
8. **Remove waste:** redundant checkouts (shallow clone `depth 1`), duplicate builds, unnecessary sleeps; fail fast.

How pipeline caching works:

- A cache is saved under a **key**, usually built from the OS plus a hash of the lock file (`package-lock.json`, `pom.xml`, `requirements.txt`). Same lock file -> same key.
- **Cache hit:** the key matches exactly, the folder is restored and the download/install step is skipped or near-instant.
- **Cache miss:** no match, so the step runs fully and the cache is saved at the end for next time.
- **Restore keys** are fallback prefixes (e.g. `npm | Linux`): on a miss, the most recent partially matching cache is restored, so only the changed dependencies download.
- Tools: GitHub `actions/cache` (or `setup-node` with `cache: npm`), Azure DevOps `Cache@2` task, Jenkins with a persistent volume or a cache plugin, GitLab `cache:key:files`.
- **Docker layer cache:** each Dockerfile instruction is a layer; if the instruction and its inputs did not change, the layer is reused. On ephemeral agents, store it in a registry (`--cache-from/--cache-to type=registry`) or `type=gha`.
- Never cache build outputs that should be fresh, and never cache secrets.

9. **Deploy faster:** only changed services, proper readiness probe tuning instead of fixed waits.

Typical result: 40 min -> ~10-12 min (70%+ faster).

Example:

```
# GitHub Actions dependency cache
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ runner.os }}-${{ hashFiles('package-lock.json') }}
    restore-keys: npm-${{ runner.os }}-

# Azure DevOps Cache@2
- task: Cache@2
  inputs:
    key: 'npm | "$(Agent.OS)" | package-lock.json'
    restoreKeys: 'npm | "$(Agent.OS)"'
    path: $(npm_config_cache)

# Docker layer cache via registry
docker buildx build --cache-from type=registry,ref=$ECR/app:cache \
  --cache-to type=registry,ref=$ECR/app:cache,mode=max -t $ECR/app:$TAG --push .

stage('Checks') {
  parallel {
    stage('Unit')  { steps { sh 'mvn -B -o test' } }
    stage('Lint')  { steps { sh 'npm run lint' } }
    stage('Scan')  { steps { sh 'trivy fs --exit-code 1 .' } }
  }
}
checkout([$class: 'GitSCM', extensions: [[$class: 'CloneOption', shallow: true, depth: 1]]])
```

When one pipeline builds many images (for example 40 images in 18 minutes):

1. **Build only what changed:** path filters per service (`dorny/paths-filter`, Jenkins `changeset`, or a script diffing `git diff --name-only origin/main...HEAD`). Usually only 2-3 of the 40 images need rebuilding.
2. **Parallelize:** a matrix job (GitHub Actions `strategy.matrix`, Jenkins `parallel`), one image per runner, instead of 40 sequential builds.
3. **Remote layer cache:** BuildKit with `--cache-from/--cache-to type=registry` (or `type=gha`), so a fresh runner reuses layers from the last build.
4. **Cache mounts** for package managers: `RUN --mount=type=cache,target=/root/.npm npm ci`.
5. **Shared base image** with the heavy OS and dependencies, built once a week, so service images only add their own code.
6. **Order Dockerfile layers** so dependency files are copied before source code, and use a good `.dockerignore`.
7. Bigger or ARM runners, and avoid `--no-cache` in CI.

```
docker buildx build   --cache-from type=registry,ref=$REG/meter-api:buildcache   --cache-to   type=r
    egistry,ref=$REG/meter-api:buildcache,mode=max   -t $REG/meter-api:$GIT_SHA --push services/mete
    r-api
```

> ***What to say to the interviewer:*** *I start by measuring stage durations to find the bottleneck, then add dependency and Docker layer caching, run tests and scans in parallel, build only changed services, optimize the Dockerfile, and move heavy e2e tests to later stages or nightly runs. Using warm agents and shallow clones as well, this typically cuts a 40-minute pipeline to around 10 minutes, 70 percent or more faster.*

Q57. How would you implement blue/green deployment with rollback on Azure using Terraform and pipelines?

> **In simple words:** Terraform builds both stages (Blue and Green) once. The pipeline puts the new show on the idle stage, tests it, then flips the spotlight. If anything looks wrong, flip it back.

Detailed answer

Option A - Azure App Service deployment slots (simplest):

1. Terraform creates the App Service with a `staging` slot.
2. Pipeline deploys the new build to the `staging` slot.
3. Run smoke tests against the staging slot URL.
4. **Swap** staging <-> production (warm-up happens before swap, no cold start).
5. Rollback = swap again (old version is still in the staging slot).

Option B - AKS: Terraform manages infra; two Deployments (`app-blue`, `app-green`); pipeline switches Service selector or Application Gateway / Front Door backend weights.

Pipeline design (Azure DevOps multi-stage YAML): Build -> Deploy Green -> Test -> Approval (environment check) -> Swap -> Monitor -> auto swap back if health/alerts fail.

Example:

```
# Terraform
resource "azurerm_linux_web_app_slot" "staging" {
  name           = "staging"
  app_service_id = azurerm_linux_web_app.app.id
  site_config {}
}

# azure-pipelines.yml (deploy stage)
- task: AzureWebApp@1
  inputs: { azureSubscription: 'sc-prod', appName: 'orders-web',
            deployToSlotOrASE: true, resourceGroupName: 'rg-prod',
            slotName: 'staging', package: '$(Pipeline.Workspace)/drop/*.zip' }
- script: curl -fsS https://orders-web-staging.azurewebsites.net/health
- task: AzureAppServiceManage@0
  inputs: { azureSubscription: 'sc-prod', Action: 'Swap Slots', WebAppName: 'orders-web',
            ResourceGroupName: 'rg-prod', SourceSlot: 'staging' }

# Rollback = same swap task again
```

What to say to the interviewer: *I use Terraform to create the environment including a staging deployment slot, or two blue and green deployments on AKS. The pipeline deploys to the idle slot, runs smoke tests, waits for the environment approval, and swaps; if health checks or alerts fail after the swap, a rollback stage swaps back, which is instant because the old version is still warm.*

Q58. Compare deployment strategies: in-place, rolling, immutable, traffic splitting (canary) and blue/green.

In simple words: Replacing the seats in a cinema: in-place closes the whole cinema; rolling fixes one row at a time; immutable builds new rows next to the old ones; traffic splitting lets a few guests try the new seats first; blue/green builds a second cinema and moves everyone over at once.

Detailed answer

This is the umbrella comparison; Blue-Green and Canary are explained in detail in Q12.

- **In-place (all at once):** stop the app on existing servers, deploy new version, start. Cheapest, but downtime and a failure hits everyone.
- **Rolling:** update a batch of instances at a time (e.g. 25%); remaining batches keep serving. A bad version affects only the current batch until you stop.
- **Immutable:** launch a brand-new set of instances with the new version (e.g. a temporary Auto Scaling group), shift to them once healthy, terminate the old ones. Never modifies running servers.
- **Traffic splitting (canary):** new instances receive a small % of traffic first, increased if metrics are healthy.
- **Blue/green:** full parallel environment, switch all traffic at once, switch back to roll back.

Strategy	Impact if failed	Deploy time	Zero downtime	DNS change	Rollback	Code deployed to
In-place (all at once)	Downtime	1x	No	No	Redeploy	Existing instances
Rolling	Single batch impacted	2x	Yes	No	Redeploy	Existing instances
Immutable	Minimal	4x	Yes	No	Redeploy	New instances

Strategy	Impact if failed	Deploy time	Zero downtime	DNS change	Rollback	Code deployed to
Traffic splitting (canary)	Minimal	4x	Yes	No	Re-route traffic and terminate new instances	New instances
Blue/green	Minimal	4x	Yes	Yes (see note)	Switch back	New instances

Accuracy note: the "DNS change = Yes" for blue/green comes from the classic **Elastic Beanstalk "Swap environment URLs" (CNAME swap)** style. If you switch with an **ALB listener / target-group** change, weighted target groups, or a **Kubernetes Service selector**, there is **no DNS change** - and switching is faster because it avoids DNS caching/TTL delays. "Deploy time" is relative (1x = fastest), because immutable, canary and blue/green all launch new capacity.

Which one to choose:

- **In-place:** dev/test environments or tools where a short outage is acceptable and cost matters most.
- **Rolling:** default for most stateless services (Kubernetes RollingUpdate), when brief mixed versions are fine.
- **Immutable:** when you want a clean, known-good fleet every time and fast safe rollback without patching servers (AMI-based EC2 fleets).
- **Traffic splitting (canary):** high-traffic user-facing services with good metrics, where you want the smallest blast radius.
- **Blue/green:** big or risky releases needing instant full rollback and pre-switch testing, when you can afford double capacity briefly.

Example:

```
# Elastic Beanstalk deployment policy (.ebextensions/deploy.config)
option_settings:
  aws:elasticbeanstalk:command:
    DeploymentPolicy: Immutable        # AllAtOnce | Rolling | RollingWithAdditionalBatch
                                       # | Immutable | TrafficSplitting
  aws:elasticbeanstalk:trafficsplitting:
    NewVersionPercent: "10"
    EvaluationTime: "5"

# Classic EB blue/green = CNAME swap (this is the DNS change)
aws elasticbeanstalk swap-environment-cnames \
  --source-environment-name orders-blue --destination-environment-name orders-green
```

> ***What to say to the interviewer:*** *In-place is cheapest but causes downtime, rolling updates batches so only one batch is at risk, immutable launches a fresh fleet and swaps to it, traffic splitting sends a small percentage to new instances first, and blue/green runs a full parallel environment and switches all traffic. Blue/green only needs a DNS change in the classic Elastic Beanstalk CNAME-swap style - with an ALB target-group switch or a Kubernetes Service selector there is no DNS change. I use rolling as the default, canary for high-traffic services, and blue/green or immutable when I need instant, clean rollback.*

Q59. How do you implement environment-specific CI/CD pipelines for 10+ microservices, and how do you scale and manage them?

In simple words: Do not write 10 services x 3 environments = 30 separate recipes. Write one master recipe with blanks to fill in ("which service?", "which environment?"), and keep each environment's settings on its own small card.

Detailed answer

This question is about pipeline design; the deploy/rollback mechanics at 100+ service scale are covered in Q54.

1. **One pipeline definition, many services (templating):** Jenkins **shared library** step like `microservicePipeline(app: 'orders')` so each repo Jenkinsfile is 2-3 lines; GitHub Actions **reusable workflows** (`workflow_call`) in a central repo; GitLab `include:` of central templates. Change the template once -> all services get the fix. Version the template (tags) so teams upgrade safely.
2. **Environments are parameters, not copies:** the same pipeline takes `ENV=dev|staging|prod`; differences live in config (Helm values per env, Kustomize overlays, per-env secrets from Secrets Manager/Key Vault), never in separate pipeline code.
3. **Build once, promote:** CI builds and scans the image once; CD promotes the same tag through environments.
4. **GitOps for deployment:** an ArgoCD **ApplicationSet** generates one Application per service per environment from the config repo layout; per-env overlays hold replica counts, resources and URLs.
5. **Promotion = PR or tag bump:** automatic to dev on merge, PR to staging/prod folders with required reviewers, or an automated image-tag bump (Argo CD Image Updater, a pipeline step).
6. **Guardrails per environment:** GitHub **environment protection rules** (required reviewers, wait timers, branch restrictions), environment-scoped secrets, separate AWS accounts/roles per env, prod sync windows.
7. **Scale the runners:** ephemeral agents (Kubernetes pod agents, ARC runners), path filters so only changed services build.

Example:

```
# Central repo: .github/workflows/service-pipeline.yml (reusable)
on:
  workflow_call:
    inputs:
      service:     { type: string, required: true }
      environment: { type: string, required: true }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}      # protection rules apply per env
    steps:
      - uses: actions/checkout@v4
      - run: |
          helm upgrade --install ${{ inputs.service }} charts/service \
            -f envs/${{ inputs.environment }}/values.yaml --atomic

# Each service repo: a few lines
jobs:
  prod:
    uses: org/ci-templates/.github/workflows/service-pipeline.yml@v3
    with: { service: orders, environment: prod }
    secrets: inherit

# GitOps layout
apps/orders/base/   apps/orders/overlays/dev/   apps/orders/overlays/prod/
```

What to say to the interviewer: *I keep one templated pipeline - a Jenkins shared library or a GitHub reusable workflow - that every service calls with its name and target environment, so environments are parameters and their differences live in per-env Helm values or Kustomize overlays and secrets. CI builds the image once, and deployment is GitOps with ArgoCD ApplicationSets, where promotion is a PR or image-tag bump. Environment protection rules, per-env accounts and ephemeral runners keep it safe and scalable.*

Q60. How would you design a GitOps pipeline for a microservices application? Which tools, what flow, and how do you roll back?

Also asked as: How does Argo CD work in CI/CD, and why use it? / How do you design GitOps for multiple teams with independent releases?

In simple words: Git becomes the single "master plan" of what should run in the cluster. Nobody touches the cluster directly - you change the plan through a reviewed PR, and a robot inside the cluster (Argo CD) keeps making reality match the plan. To undo, you undo the plan.

Detailed answer

For deploying and rolling back 100+ services at scale see Q54; this answer focuses on the GitOps design itself.

How Argo CD works and why use it: Argo CD is a Kubernetes controller that watches Git repos. For each Application it renders the manifests (Helm, Kustomize or plain YAML), compares them with the live cluster, and marks the app Synced/OutOfSync and Healthy/Degraded. With auto-sync it applies changes itself. In CI/CD it takes over the CD half: CI only builds and pushes the image and updates Git; Argo CD deploys. Why: no cluster credentials in CI (pull model), Git is the audit trail, drift is visible and self-healed, rollback is a revert, and one UI shows every app across clusters.

Flow:

1. **App repo CI:** build, test, scan, then push the image with an **immutable tag or digest** (`1.4.0-a1b2c3d`, `@sha256:...`).
2. **Config repo update:** a CI bot opens a PR (or Argo CD Image Updater commits) that bumps the tag in the Helm values or Kustomize overlay for that environment. Dev can auto-merge; staging and prod need reviewers.
3. **Sync:** Argo CD (or Flux) running in the cluster detects the commit and applies it. **ApplicationSets** generate Applications for many services/envs; **sync waves** order things (CRDs and config first, then apps); **health checks** decide if the sync is Healthy.
4. **Progressive delivery:** Argo Rollouts canary/blue-green with analysis on Prometheus metrics, auto-abort on bad metrics.
5. **Rollback:** `git revert` the config commit - Git is the source of truth. Argo CD's "rollback to previous sync" works only if auto-sync is disabled; with auto-sync on, Argo re-applies whatever Git says, so revert in Git.

Also design in:

- **Drift self-heal:** `selfHeal: true` undoes manual `kubectl edit` changes; `prune: true` removes deleted resources.
- **Security:** Argo CD **SSO + RBAC** (AppProjects restrict which repos/namespaces/clusters a team can deploy to); CI never needs cluster credentials (pull model).
- **Secrets:** never plain secrets in Git - use **External Secrets Operator** (pulls from AWS Secrets Manager/Vault) or **SOPS**/Sealed Secrets (encrypted in Git).
- **Notifications:** Argo CD Notifications to Teams/Slack on sync failure or degraded health.

```
Dev push -> [App repo CI: build/test/scan] -> push image (immutable tag) -> ECR
                                  |
                                  v
                PR: bump tag in [Config repo: envs/dev|staging|prod]
                                  |  (review + merge)
                                  v
             [Argo CD in cluster] --pull + sync--> Kubernetes
                                  |                     |
                       self-heal drift            Argo Rollouts canary
                                  |                     |
                rollback = git revert <---- analysis fails -> auto abort
```

Example:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: orders-prod
  namespace: argocd
spec:
  project: payments-team
  source:
    repoURL: https://github.com/org/gitops-config.git
    targetRevision: main
    path: apps/orders/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: orders
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions: [CreateNamespace=true]
    retry: { limit: 3, backoff: { duration: 10s, factor: 2 } }

# Rollback
git revert 3f2a1bc && git push     # Argo CD syncs the previous version
```

GitOps for many teams with independent releases:

- **App-of-Apps:** one root Argo CD Application points to a folder of child Application manifests, so adding a service means adding one file.
- **ApplicationSets:** generate Applications automatically. A git directory generator creates one app per folder `teams/*/services/*`; a cluster generator deploys the same app to every cluster.
- **AppProject per team:** restricts which repos a team can deploy from, which namespaces or clusters it can deploy to, and which resource kinds are allowed, with RBAC mapped to SSO groups. One team cannot touch another team's namespace.
- **Repo layout:** a platform or infra repo owned by the platform team (ingress, cert-manager, monitoring), plus per-team config repos or folders. Each team merges and releases on its own schedule.
- **Guardrails:** sync windows (no prod syncs during business-critical hours), required PR reviews on prod folders, and progressive delivery with Argo Rollouts.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: team-services, namespace: argocd }
spec:
  generators:
    - git:
        repoURL: https://github.com/my-org/deploy-config.git
        revision: main
        directories: [{ path: "teams/*/services/*" }]
  template:
    metadata: { name: "{{path[1]}}-{{path.basename}}" }
    spec:
      project: "{{path[1]}}"              # AppProject per team
      source:
        repoURL: https://github.com/my-org/deploy-config.git
        targetRevision: main
        path: "{{path}}"
      destination: { server: https://kubernetes.default.svc, namespace: "{{path[1]}}" }
      syncPolicy: { automated: { prune: true, selfHeal: true } }
```

What to say to the interviewer: *The app repo CI builds, tests and scans, then pushes an image with an immutable tag, and a bot or Argo CD Image Updater bumps that tag in the config repo per environment through a PR. Argo CD in the cluster pulls and syncs using ApplicationSets, sync waves and health checks, with Argo Rollouts for canary analysis. Rollback is a git revert because Git is the source of truth, and the design includes self-heal for drift, SSO and RBAC through AppProjects, and secrets through External Secrets or SOPS rather than plain secrets in Git.*

Q61. How do you handle databases in blue-green deployments? With critical data, how do you keep the database consistent and avoid issues during switchover?

In simple words: Blue and green are two kitchens, but they usually share ONE fridge (the database). So any change to the fridge must work for both kitchens at the same time - you add a new shelf before anyone uses it, and only remove the old shelf when nobody needs it anymore.

Detailed answer

What to do when a migration step fails mid-deploy is covered in Q75; this answer is about designing the database side of blue-green.

Core idea: **blue and green normally share one database.** Only the application layer is duplicated, so the schema must support both versions during the switch and during a rollback.

1. **Backward-compatible schema changes with expand -> migrate -> contract**, spread across releases:

- Release 1 (expand): add the new column/table, nullable or with a default; old code ignores it.
- Release 2 (migrate): new code writes both old and new, backfill existing rows in batches, then read from new.
- Release 3 (contract): once no running version uses the old column, drop it.

2. **Never make destructive changes (drop/rename column, change type) in the same release** as the code that needs them - blue would break the moment the migration runs, and rollback would be impossible.

3. **Feature flags:** ship the code dark, turn the new behaviour on after the switch, and turn it off instead of rolling back.
4. **Avoid conflicting dual writers:** blue and green must not write the same data in incompatible formats (e.g. green writes JSON v2 that blue cannot parse). Keep formats versioned and readable by both.
5. **Switchover hygiene:** drain connections, let long-running transactions and background jobs finish, and make sure only one colour runs schedulers/queue consumers that must not run twice.
6. **Rollback implication:** data written by green must still be readable by blue. If it is not, switching back is not a real rollback.
7. **If you must use separate databases:** keep them in sync with replication/CDC (AWS DMS, PostgreSQL logical replication), plan a short **write freeze**, verify replication lag = 0 and row counts/checksums, then cut over; decide in advance how to handle writes if you roll back (reverse replication).
8. **RDS Blue/Green Deployments** for engine upgrades or parameter changes: AWS creates a green copy kept in sync by replication and performs a managed switchover (typically about a minute), renaming endpoints so apps do not change connection strings.

Example:

```
-- Release 1: expand (safe for blue and green)
ALTER TABLE customers ADD COLUMN email_normalized VARCHAR(255);

-- Release 2: backfill in batches while code dual-writes
UPDATE customers SET email_normalized = lower(email)
WHERE email_normalized IS NULL AND id BETWEEN 1 AND 10000;

-- Release 3: contract (only after blue with old code is gone)
ALTER TABLE customers DROP COLUMN email_old;

# RDS managed blue/green for an engine upgrade
aws rds create-blue-green-deployment --blue-green-deployment-name pg16-upgrade \
  --source arn:aws:rds:ap-south-1:123456789012:db:orders-db --target-engine-version 16.4
aws rds switchover-blue-green-deployment \
  --blue-green-deployment-identifier bgd-abc123 --switchover-timeout 300
```

What to say to the interviewer: *In blue-green both colours usually share one database, so every schema change must be backward compatible - I use expand, migrate, contract across separate releases and never drop or rename columns in the same release as the code that needs it. I use feature flags, avoid incompatible dual writes, drain connections and long transactions on switchover, and make sure data written by green is readable by blue so rollback stays safe. If separate databases are unavoidable I use replication or CDC like DMS with a write freeze and lag check at cutover, and for engine upgrades I use RDS Blue/Green Deployments with managed switchover.*

Scenario-Based

Q62. The Jenkins controller goes down while builds are running on agents. What happens and what do you do?

In simple words: If the manager faints, the workers keep their tools in hand for a little while, but nobody is recording their work or giving next instructions. When the manager wakes up, Pipeline jobs can often pick up where they left off; Freestyle jobs are simply lost.

Detailed answer

What happens:

- Agents lose their connection (remoting channel) to the controller.
- **Freestyle jobs** running on agents fail/abort - their state lived on the controller.
- **Declarative/Scripted Pipelines are durable**: state is checkpointed to disk on the controller. After restart, pipelines **resume**; an `sh` step already running on an agent can continue if the agent and workspace survive (durable task). Some steps may still fail and need a rerun.
- No new builds are scheduled; webhooks received during downtime are missed (need re-scan / redeliver).

What I do:

1. Restore the controller: check the service/pod (`systemctl status jenkins` or `kubectl get pod`), logs, disk, memory (OOM is common), then restart.
2. Check which builds resumed vs failed; rerun failed ones; redeliver missed GitHub webhooks or trigger a branch re-scan.
3. Make sure a half-finished deploy did not leave prod inconsistent (check Helm/rollout status).

Prevent:

- Run the controller on Kubernetes/EC2 with persistent `JENKINS_HOME` (EBS/EFS) so it auto-restarts; back up `JENKINS_HOME` (thinBackup / volume snapshots).
- Configuration as Code (JCasC) + jobs in Git so a new controller can be rebuilt fast.
- Monitor the controller (Prometheus plugin, heap, disk) and set pipeline `durabilityHint` appropriately.
- Deploy steps should be idempotent (`helm upgrade --install --atomic`) so a rerun is safe.

Example:

```
kubectl -n jenkins get pods
kubectl -n jenkins logs jenkins-0 --previous | tail -50   # why did it die? OOMKilled?
kubectl -n jenkins describe pod jenkins-0 | grep -i -A3 "last state"

options { durabilityHint('MAX_SURVIVABILITY') }   // strongest resume guarantees
```

What to say to the interviewer: *Agents lose their connection; Freestyle jobs are lost, but Pipelines are durable and usually resume after the controller restarts, though some steps may need a rerun. I restore the controller, check resumed versus failed builds, rerun and redeliver missed webhooks, and verify no deployment was left half-done. To prevent it we run Jenkins with persistent storage and auto-restart, JCasC for fast rebuild, backups and monitoring.*

Q63. The Jenkins master (controller) is running out of disk space. What actions would you take?

Also asked as: If Jenkins storage is full, how would you troubleshoot it?

In simple words: Jenkins keeps every old receipt, log and leftover box forever unless told not to. First throw away the obvious junk to get breathing room, then set rules so the junk does not pile up again.

Detailed answer

Immediate (get space back safely):

1. Find what is big: `du` on `JENKINS_HOME` - usually `jobs/*/builds` (old build logs + artifacts), `workspace/`, `caches`, `logs`, or Docker images if builds run on the controller.
2. Delete old builds via Script Console or job config (not by randomly deleting folders while Jenkins runs).
3. Wipe old workspaces (`workspace/`), clean `/tmp`, rotate `jenkins.log`, `docker system prune` if Docker is on the controller.
4. If still critical, expand the EBS volume (online resize) to avoid an outage.

Long term:

- `buildDiscarder(logRotator(numToKeepStr: '20', artifactNumToKeepStr: '5'))` on every job (enforce via shared library).
- **Don't build on the controller**: set controller executors to 0; builds on ephemeral agents.
- Store artifacts in S3/Nexus/Artifactory, not in Jenkins (`archiveArtifacts` only for small reports).
- `cleanWs()` in `post { always }`.
- Disk usage alert at 80% (Prometheus node exporter / CloudWatch agent).

Example:

```
df -h /var/lib/jenkins
du -sh /var/lib/jenkins/* | sort -rh | head
du -sh /var/lib/jenkins/jobs/*/builds | sort -rh | head

// Script Console: keep only last 20 builds for every job
Jenkins.instance.getAllItems(Job).each { job ->
  job.builds.drop(20).each { it.delete() }
}

options { buildDiscarder(logRotator(numToKeepStr: '20', artifactNumToKeepStr: '5')) }
```

What to say to the interviewer: *I first find what is consuming space in JENKINS_HOME - usually old builds, artifacts and workspaces - and safely delete old builds through the Script Console, clean workspaces and logs, and expand the volume if it is critical. Long term I enforce build discarders on all jobs, run zero executors on the controller, store artifacts in S3 or Nexus, clean workspaces after each build, and alert on disk at 80 percent.*

Q64. Build artifacts are not getting uploaded to the artifact repository, or Jenkins jobs fail randomly at the artifact upload step. How do you investigate?

Also asked as: Jenkins jobs are randomly failing at the artifact upload step. What layers would you check?

In simple words: Uploading a parcel can fail because of the sender (the build), the road (network), the gatekeeper (credentials/permissions), or the warehouse (the repository is full or broken). Check each layer in order.

Detailed answer

Layers to check:

1. **Build layer:** did the artifact actually get produced? Wrong path/pattern, artifact built in another stage's workspace or another agent (use `stash/unstash`).
2. **Credentials/auth:** expired token or password, ECR token expired (12h), IAM role missing `ecr:PutImage`/`s3:PutObject`, KMS permission for encrypted bucket. Intermittent -> token expiring mid-build or different agents with different roles.
3. **Network:** DNS resolution, proxy, firewall/SG, NAT gateway limits, timeouts on large files, TLS certificate errors. Random failures often = specific agents in a subnet without the right route.
4. **Repository side:** Nexus/Artifactory disk full, repo read-only, version already exists (release repo disallows redeploy, ECR tag immutability), rate limiting (429), 5xx during repo maintenance.
5. **Agent resources:** agent disk full (cannot stage upload), OOM killing the upload process.
6. **Size/timeouts:** very large artifacts hit proxy or client timeouts - use multipart upload, increase timeouts.

Fix pattern: read the exact error code (401/403 auth, 404 path, 409 conflict/exists, 413 too large, 429 rate limit, 5xx server), correlate failures with agent name and time, add `retry` with backoff only for transient 5xx/network errors, and add monitoring on the repository.

Example:

```
sh 'ls -lh target/*.jar'                                  # artifact exists?
sh 'aws sts get-caller-identity'                          # which identity is used?
sh 'curl -sv https://nexus.example.com/service/rest/v1/status 2>&1 | tail -20'

retry(3) {
  sh 'mvn -B deploy -DskipTests'                          # transient errors only
}
// Correlate: failed builds -> which agent? env.NODE_NAME
echo "Running on ${env.NODE_NAME}"
```

What to say to the interviewer: *I check layer by layer: whether the artifact was really produced at that path, then credentials and IAM permissions including expiring tokens, then network - DNS, proxy, security groups and timeouts, then the repository itself for disk, immutability conflicts, rate limits or 5xx errors, and finally agent disk and memory. For random failures I correlate them with the agent name and time, read the HTTP status code, and only add retries with backoff for genuine transient errors.*

Q65. A Jenkins pipeline fails randomly after long runs. How do you find the root cause?

In simple words: If a marathon runner collapses only after 2 hours, the problem is usually stamina (memory), running out of water (disk), or the race timer (timeouts and expiring passes) - not their shoes.

Detailed answer

Common causes of "fails only on long runs":

- **Expired credentials:** AWS STS session (1h default), ECR token, Kubernetes token, OAuth token expiring mid-build.
- **Timeouts:** pipeline `timeout`, agent connection timeouts, load balancer/proxy idle timeouts cutting a long quiet `sh` step, SSH keepalive.
- **Agent problems:** Kubernetes pod agent evicted/OOMKilled, spot instance reclaimed, agent disk filling with logs/temp files, agent disconnected (ping thread timeout).
- **Controller pressure:** heap exhaustion or GC pauses under long heavy logs.
- **Resource leaks:** test processes, Docker containers, file descriptors piling up.
- **External dependencies:** rate limits, DB connections timing out.

How to find it:

1. Collect several failed runs: same stage? same agent? same duration (e.g. always ~60 min = token expiry)?
2. Read the error at the exact failure point and the agent log (`Manage Jenkins -> Nodes -> agent -> Log`).
3. Check agent pod events (`kubectl describe pod` - OOMKilled/Evicted) or EC2 spot interruption notices.
4. Check controller logs and JVM heap/GC metrics.
5. Reproduce with a debug run; add `timestamps()` to see where time goes.

Fixes: refresh credentials per stage (assume role with longer duration or re-login), right-size agent resources, use on-demand agents for long jobs, add keepalive output for quiet steps, split long jobs into stages or parallel shards, cleanup in `post`.

Example:

```
kubectl -n jenkins get events --sort-by=.lastTimestamp | grep -i -E "oom|evict|kill"
aws sts assume-role --role-arn $ROLE --role-session-name ci --duration-seconds 14400
stage('Push') {
  steps { sh 'aws ecr get-login-password | docker login -u AWS --password-stdin $ECR' } // re-login
}
```

What to say to the interviewer: *I look for a pattern across failures - same stage, same agent, or same elapsed time, since a failure around 60 minutes usually means an expired STS or registry token. Then I check agent logs and pod events for OOMKilled, eviction or spot interruption, controller heap, and network idle timeouts. Fixes include refreshing credentials per stage, right-sizing agents, on-demand agents for long jobs, keepalive output, and splitting the job into smaller parallel stages.*

Q66. The Jenkins pipeline runs, but the build does not actually happen. What could be wrong?

In simple words: The oven turned on but no cake came out. Maybe the recipe said "only bake on Sundays" (a when condition), there was no cook available (no agent), or someone rang the wrong kitchen's bell (the webhook hit a different job).

Detailed answer

Possible causes and checks:

- `when` **conditions / branch filters skip stages:** `when { branch 'main' }` but the build is on `feature/x` or a PR (`BRANCH_NAME` is `PR-12`); stages show grey "skipped" in Stage View.
- **No agent / executors:** wrong `agent { label 'docker' }` label, agents offline, all executors busy - build sits in queue "Waiting for next available executor".
- **Webhook or polling hits the wrong job:** repo URL in job does not exactly match the webhook payload (https vs ssh, `.git`), branch spec `*/master` vs `main`, so Jenkins "sees" no changes.
- **Empty changeset:** Poll SCM found no new commits, or a path/`changeset` condition excludes the changed files; monorepo change detection skipping every service.
- **Cached/stale workspace:** old checkout, `git diff` against wrong base, build tool says "up-to-date" and skips.
- **Script approval / sandbox:** Groovy method not approved (`Scripts not permitted to use method...`) - pipeline stops early.
- **Parameters:** default parameter values like `SKIP_BUILD=true` or empty `TAG`; first run of a parameterized pipeline uses defaults.
- **Script bug:** `sh` returns 0 even though it did nothing (`set -e` missing, `|| true`), wrong working directory (`dir()`).

Debug order: open Stage View (which stages skipped) -> console log -> `echo` key variables (`BRANCH_NAME`, `CHANGE_ID`, params) -> check queue/agents -> check webhook deliveries and job SCM config -> Script Approval page.

Example:

```
stage('Debug') {
  steps {
    echo "BRANCH=${env.BRANCH_NAME} PR=${env.CHANGE_ID} NODE=${env.NODE_NAME}"
    echo "PARAMS=${params}"
    sh 'git log -1 --oneline && git status'
  }
}
sh '''
  set -euo pipefail          # fail loudly instead of silently doing nothing
  make build
'''
```

What to say to the interviewer: *I open the Stage View to see which stages were skipped - often a when branch condition, or a PR build where the branch name is different. Then I check the queue for missing agent labels or executors, the webhook and job SCM config for repo or branch mismatches, empty changesets or stale workspaces, pending script approvals, and default parameter values. Echoing the branch, parameters and node, and using set -e in shell steps, makes the cause obvious*

quickly.

Q67. Jenkins credentials were exposed accidentally. What is your immediate action plan?

In simple words: If you lose your house key, you do not first investigate how you lost it - you change the locks right now, then check if anyone entered, then figure out how it happened.

Detailed answer

Immediate (minutes):

1. **Revoke/rotate** the exposed secret at the source: deactivate the AWS access key, regenerate GitHub token, change DB password, rotate API key. Update the Jenkins credential with the new value.
2. **Contain:** if a cloud key, attach a deny-all policy or disable the IAM user; revoke active sessions (`aws iam` revoke older sessions for roles).
3. **Remove the exposure:** delete the log/build output, purge from Git history (`git filter-repo` / BFG) and force push, remove public paste/screenshot; note: Git history purging does not un-leak it - rotation is what matters.

Investigate (hours):

4. Check **CloudTrail / audit logs** for usage of the key since the exposure time (unknown IPs, new IAM users, EC2 instances in odd regions, S3 downloads).
5. Check Jenkins audit trail: who could see it, which jobs used it.
6. Clean up any attacker-created resources.

Prevent:

7. Mask secrets (`withCredentials`), never echo; enable secret scanning (gitleaks, GitHub push protection); replace static keys with IAM roles/OIDC; restrict credential permissions by folder; blameless postmortem.

Example:

```
aws iam update-access-key --user-name ci-user --access-key-id AKIA... --status Inactive
aws iam create-access-key --user-name ci-user          # only if a key is truly needed
aws cloudtrail lookup-events --lookup-attributes \
  AttributeKey=AccessKeyId,AttributeValue=AKIA... --max-results 50
git filter-repo --replace-text secrets-to-remove.txt
```

What to say to the interviewer: *First I revoke or rotate the credential at the source immediately and update Jenkins with the new one, then remove the exposure from logs and Git history, knowing rotation is what actually protects us. Next I check CloudTrail and audit logs for any misuse since the exposure and clean up anything suspicious. Finally I prevent recurrence with masking, secret scanning with push protection, IAM roles instead of static keys, and a blameless postmortem.*

Q68. CI/CD logs show flaky tests breaking the pipeline. How do you handle them?

In simple words: A flaky test is a smoke alarm that goes off randomly. If you ignore it, people stop trusting all alarms. Take it off the wall, fix it, then put it back - do not just remove the batteries forever.

Detailed answer

1. **Confirm it is flaky:** same commit passes and fails on rerun. Track failure rates per test (JUnit history, Flaky Test Handler plugin, test analytics).
2. **Quarantine:** move known flaky tests into a separate non-blocking suite/tag so the main pipeline stays trustworthy - with a ticket and an owner, and a deadline.
3. **Find the root cause** - common ones:

- timing: `sleep` instead of waiting for a condition, async code, timeouts too tight.
- shared state: tests depend on order, shared DB rows, fixed ports, parallel tests clashing.
- external dependencies: real network calls, third-party APIs -> use mocks/test containers.
- environment: time zones, random seeds, low agent resources.

4. **Fix and bring back** into the blocking suite.
5. Limited `retry` for a known unstable integration step is okay short-term, but never blanket-retry the whole test stage - it hides real bugs.
6. Make metrics visible: flaky count on a dashboard, a "no new flaky tests" rule.

Example:

```
# Run tests in random order to expose order dependence
mvn test -Dsurefire.runOrder=random
pytest -p randomly --count=20 tests/test_orders.py   # repeat to confirm flakiness

stage('Tests') {
  steps { sh 'mvn -B test -Dgroups=!quarantine' }          // blocking suite
}
stage('Quarantined tests') {
  steps { catchError(buildResult: 'SUCCESS', stageResult: 'UNSTABLE') {
    sh 'mvn -B test -Dgroups=quarantine' } }               // non-blocking
}
```

> ***What to say to the interviewer:*** *I first confirm flakiness by rerunning the same commit and tracking failure rates, then quarantine the flaky tests into a non-blocking suite with an owner and ticket so the main pipeline stays trustworthy. The root cause is usually timing, shared state, test ordering or external dependencies, which we fix with proper waits, isolation and mocks before bringing the test back. I avoid blanket retries because they hide real bugs.*

Q69. A GitHub Actions workflow is stuck in "pending" or "queued" state. How do you troubleshoot it?

> **In simple words:** The job is waiting in line for a worker. Either no worker with the right badge exists, all workers are busy, or a gate (approval, concurrency lock) is holding it back.

Detailed answer

Checks in order:

1. **GitHub status page** - Actions outage or degraded service.
2. `runs-on` **label mismatch** - job asks for `self-hosted, linux, gpu` but no runner has all those labels -> waits forever ("Waiting for a runner to pick up this job").
3. **Self-hosted runners offline/busy** - Settings -> Actions -> Runners: offline, runner service stopped, runner in a group not allowed for this repo, autoscaler (ARC on Kubernetes) not scaling

- check controller logs and pod pending reasons.

4. **Concurrency groups** - `concurrency:` with an older run in progress holds new ones pending.
5. **Environment protection rules** - job targeting `environment: production` waits for required reviewers or a wait timer.
6. **Usage limits** - concurrent job limits for the plan, spending limit reached for private repos, org policy disabling actions.
7. **Workflow approval** - first-time contributors from forks need approval to run.

Fix examples: correct labels, restart runner service, scale runner pool, cancel the stale run holding the concurrency lock, approve the deployment, raise limits.

Example:

```
jobs:
  deploy:
    runs-on: [self-hosted, linux, x64]      # must match a registered runner's labels
    environment: production                 # may wait for reviewers
    concurrency:
      group: deploy-prod
      cancel-in-progress: false             # new runs wait until current one finishes

# Self-hosted runner checks
sudo ./svc.sh status
kubectl -n arc-runners get pods; kubectl -n arc-systems logs deploy/arc-gha-rs-controller
gh run list --status queued ; gh run cancel <run-id>
```

> ***What to say to the interviewer:*** *A pending job is waiting for a runner or a gate. I check the GitHub status page, then whether the runs-on labels match an online runner, whether self-hosted runners or the ARC autoscaler are healthy and allowed for the repo, then concurrency groups holding the run, environment approvals, usage or spending limits, and fork approval rules. Usually it is a label mismatch or offline runners.*

Q70. Your team complains of deployment delays due to manual approvals. How would you automate this safely?

> **In simple words:** Instead of a manager personally checking every package, install good scanners and rules. If the scanner says everything is normal, the package goes through; the manager only looks at unusual ones.

Detailed answer

Replace human judgement with automated evidence:

- **Strong automated gates:** tests, coverage, SonarQube quality gate, security scans, policy checks (OPA/Conftest on manifests, Terraform plan checks) - if all pass, promotion is automatic.
- **Risk-based approvals:** auto-approve low-risk changes (config, small services, non-prod, standard changes) and keep a human approval only for high-risk ones (DB migrations, infra deletes, payment services).
- **Progressive delivery with automated rollback:** canary + metric analysis (Argo Rollouts, CodeDeploy alarms). If the canary looks bad it rolls back automatically, so the approval adds little.

- **Approve earlier in the flow:** code review on the PR is the approval; merge = approved to deploy (GitOps).
- **Deployment windows and change freezes** enforced by the tool, not by a person.
- **Audit trail:** every deploy logged with who merged, what changed, test results - satisfies compliance without waiting.
- **Make remaining approvals fast:** Teams/Slack approval buttons, approver groups instead of one person, timeouts with escalation.

Example:

```
stage('Policy check') { steps { sh 'conftest test k8s/ -p policy/' } }
stage('Approval') {
  when { expression { env.RISK == 'high' } }       // only risky changes wait
  steps { timeout(time: 4, unit: 'HOURS') {
    input message: 'Approve high-risk deploy?', submitter: 'release-managers,sre-leads' } }
}
stage('Canary') { steps { sh 'kubectl argo rollouts set image orders app=$IMAGE' } }
```

What to say to the interviewer: *I replace manual checks with automated evidence - tests, quality and security gates, and policy-as-code - and use canary deployments with automatic metric-based rollback. Then approvals become risk-based: low-risk changes promote automatically, and only high-risk changes like DB migrations need a human, via group approvers with Teams or Slack buttons. Everything is audited, so we meet compliance without the delay.*

Q71. Your production pipeline is blocked due to missing approvals and stakeholders are unreachable. What will you do?

In simple words: If the bank manager who signs cheques is on leave, you do not forge the signature. You follow the bank's backup rule - ask the deputy, or for a true emergency use the break-glass procedure that gets reviewed afterwards.

Detailed answer

1. **Assess urgency:** is it a normal release (can wait) or a production-fixing hotfix/security patch (cannot wait)?
2. **Normal release:** wait; communicate the delay; do not bypass the gate. Reschedule.
3. **Urgent:** follow the documented **escalation path** - backup approvers, on-call manager, delegate group. Approval should be to a group, not one person.
4. **Break-glass procedure** if defined: an emergency change path with a limited set of people, logged, and reviewed after (retro-approval / emergency CAB). Never share someone's login or edit the pipeline to remove the gate.
5. **Reduce risk of the change:** deploy the smallest possible fix, with canary and ready rollback, and monitoring watched live.
6. **Document everything:** who approved, why, timeline; inform stakeholders as soon as reachable.
7. **Fix the process:** add backup approvers, approval groups, SLA/timeout escalation, and define "standard changes" that can auto-approve.

Example:

```
input message: 'Approve PROD deploy?',
      submitter: 'release-managers,oncall-managers'   // group + backup, not one person

// Break-glass: separate job, restricted permission, forces an incident ticket
parameters { string(name: 'INCIDENT_ID', description: 'Required for emergency deploy') }
stage('Validate') { steps { script {
  if (!params.INCIDENT_ID?.trim()) { error 'Emergency deploy needs an incident ID' } } } }
```

What to say to the interviewer: *I first judge urgency - a normal release simply waits and I communicate the delay, because bypassing controls is not an option. For an urgent fix I follow the escalation path to backup approvers or use the documented break-glass process, deploy the smallest change with canary and rollback ready, and log everything for retrospective approval. Afterwards I fix the process with approval groups, backups and escalation timeouts.*

Q72. A pipeline deployment failed in production. How would you troubleshoot it step by step?

In simple words: First make sure customers are safe (is the old version still serving?), then read the error message, then walk backwards: was it the package, the delivery truck, or the destination?

Detailed answer

1. **Check user impact first:** is production still serving the old version (failed rollout usually leaves old pods running) or is it broken? If broken -> roll back immediately, debug later.
2. **Find the failing stage and exact error** in the pipeline log: build, push, auth, Helm/kubectl, health check timeout.
3. **Pipeline/infra layer:** credentials/role expired, kubeconfig/context wrong, registry unreachable, Helm release stuck in `pending-upgrade`, Terraform lock.
4. **Deployment layer (Kubernetes):** `kubectl rollout status`, `get pods`, `describe` pod events - ImagePullBackOff (wrong tag/permissions), CrashLoopBackOff (app/config), Pending (resources/quota), readiness probe failing.
5. **Application layer:** pod logs - missing env var/secret/ConfigMap, DB migration failure, dependency unreachable.
6. **Compare with the last successful deploy:** diff image tag, Helm values, config, secrets, infra changes.
7. **Fix forward or roll back**, then rerun the pipeline; verify with smoke tests and metrics.
8. **RCA + prevention:** add the missing check to the pipeline (e.g. config validation, staging parity, `helm --atomic`).

Example:

```
helm status orders-api -n prod ; helm history orders-api -n prod
kubectl -n prod rollout status deploy/orders-api
kubectl -n prod get pods -l app=orders-api
kubectl -n prod describe pod <pod> | tail -30
kubectl -n prod logs <pod> --previous
helm get values orders-api -n prod --revision 42 > old.yaml
helm get values orders-api -n prod --revision 43 > new.yaml ; diff old.yaml new.yaml
# Stuck release
helm rollback orders-api 42 -n prod
```

What to say to the interviewer: *My first step is checking user impact - if production is broken I roll back immediately. Then I find the failing stage and exact error, and check layer by layer: pipeline credentials and tooling, Kubernetes rollout and pod events for image pull, crash or pending issues, then application logs, and I diff the failed release against the last good one. After fixing or rolling back, I verify with smoke tests and add a pipeline check so it cannot recur.*

Q73. A production deployment caused downtime or an incident. How do you roll back and identify the root cause?

Also asked as: How would you handle a production incident caused by a faulty deployment?

In simple words: When a new tyre makes the car wobble on the highway, you put the old tyre back first and get the car moving. Only after that do you study why the new tyre was bad.

Detailed answer

During the incident:

1. **Declare and communicate:** incident channel, incident commander, status update to stakeholders.
2. **Correlate:** deploy time vs when errors started (deploy markers on Grafana, alert timeline). If they match, suspect the deploy.
3. **Mitigate first - roll back:** `helm rollback` / `kubectl rollout undo` / git revert in the GitOps repo / swap Blue-Green back / canary weight to 0 / turn off the feature flag. Check DB migrations are backward compatible before rolling back code.
4. **Verify recovery:** error rate, latency, and business metrics back to normal; keep watching.
5. **Freeze further deploys** for that service until understood.

After the incident (RCA):

6. **Collect evidence:** diff between versions (code, config, Helm values, dependencies), logs, traces, metrics, pipeline logs.
7. **Reproduce in staging** with the bad version.
8. **5 Whys:** why did the bug exist, and why did the pipeline not catch it (missing test, staging different from prod, no canary)?
9. **Blameless postmortem** with timeline, impact, root cause, action items with owners (add tests, canary analysis, config validation, alerts).

Example:

```
kubectl -n prod rollout undo deploy/orders-api
kubectl -n prod rollout status deploy/orders-api
git log --oneline v1.4.0..v1.4.1            # what changed between releases
git diff v1.4.0 v1.4.1 -- charts/ config/
# Grafana annotation for deploy markers
curl -X POST -H "Authorization: Bearer $GRAFANA_TOKEN" -H "Content-Type: application/json" \
  https://grafana.example.com/api/annotations -d '{"tags":["deploy","orders"],"text":"v1.4.1"}'
```

What to say to the interviewer: *I mitigate first: open an incident channel, correlate the deploy time with the error spike, and roll back through Helm, a GitOps revert, a Blue-Green switch or a feature flag, then confirm metrics recover and freeze further deploys. After that I diff the two versions, reproduce in staging, use 5 Whys to find both the bug and why the pipeline missed it, and write a blameless postmortem with owned action items like better tests or canary analysis.*

Q74. A new deployment causes a CPU spike. How will you roll back and debug?

In simple words: The new engine runs hot. Swap the old engine back in so the car keeps going, then put the new engine on a test bench to find which part overheats.

Detailed answer

Roll back / contain:

1. Confirm correlation: CPU spike starts at the deploy time; only new-version pods are hot (`kubectl top pods`, Grafana by version label).
2. If users are impacted (latency, errors, HPA maxed out, node pressure) -> **roll back immediately** (`helm rollback` / `rollout undo` / canary abort). Short-term alternative: scale out while deciding.
3. Verify CPU and latency return to baseline.

Debug the bad version (in staging or a single canary pod):

- **Diff the change:** code (new loop, regex, serialization, logging level set to DEBUG), dependency upgrade, config (thread pool, cache disabled, connection pool too small causing retries), resource limits changed (CPU throttling looks like high CPU).
- **Profile:** language profilers (async-profiler/JFR for Java, pprof for Go, py-spy), thread dumps to see hot threads; `top -H` inside the container.
- **Check traffic pattern:** retry storms, a new endpoint being called heavily, health checks too frequent.
- **Check GC:** memory leak -> constant garbage collection -> high CPU.
- **Load test** the fix before redeploying with canary.

Prevent: performance test stage in pipeline, canary analysis on CPU/latency, resource requests/limits set correctly, alerts on CPU vs baseline per version.

Example:

```
kubectl -n prod top pods -l app=orders-api --sort-by=cpu
helm rollback orders-api -n prod        # or: kubectl argo rollouts abort orders-api
kubectl -n staging exec -it <new-pod> -- top -H -b -n 1 | head -20
kubectl -n staging exec <new-pod> -- jcmd 1 Thread.print > threads.txt   # Java thread dump
# Prometheus: CPU by version
sum by (version) (rate(container_cpu_usage_seconds_total{container="orders-api"}[5m]))
```

What to say to the interviewer: *I confirm the CPU spike lines up with the deploy and only affects new-version pods, and if users are impacted I roll back or abort the canary immediately, then confirm CPU returns to baseline. I debug the new version in staging by diffing code, dependencies and config, profiling with thread dumps or a profiler, and checking for retry storms or GC pressure. Before redeploying I load-test the fix and add canary analysis on CPU and latency to the pipeline.*

Q75. A deployment to staging failed during the DB-migration step and blocked all environments. What is your rollback plan?

In simple words: A renovation stopped halfway through knocking down a wall. First stop all work so nobody makes it worse, look at exactly which bricks are already gone, then either put the wall back (restore) or finish the job carefully (fix forward). Next time, renovate one house at a time instead of making all houses wait on one.

Detailed answer

Immediate:

1. **Stop and lock:** abort the pipeline and lock promotion so nothing retries the migration or pushes this version to other environments.
2. **Check the actual DB state:** the migration tool's history table (Flyway `flyway_schema_history`, Liquibase `DATABASECHANGELOG`, Alembic `alembic_version`) - which migration failed, is it marked failed/dirty, was it partially applied? Check for leftover locks (`DATABASECHANGELOGLOCK`).
3. **Choose the recovery path:**

- Migration ran inside a transaction and rolled back cleanly -> repair the history entry, fix the script, rerun.
- Partially applied and the script is idempotent (`IF NOT EXISTS`) -> **fix forward**: correct the script and rerun.
- Partially applied and not safe -> run the **down-migration**, or **restore the snapshot** taken before the migration.

4. **Roll the app back** to the previous version only if the schema is still compatible with it.
5. **Unblock other environments:** release the lock; dev and prod should not be waiting on staging.

Prevention:

- **Expand-contract (backward-compatible) migrations:** add new columns/tables first, switch the code, remove old ones in a later release - so old and new app versions both work.
- **Automatic pre-migration snapshot/backup** step.
- **Dry-run** migrations against a prod-like copy (restored snapshot) in CI.
- **Decouple migrations into their own job/stage** per environment, run before the app deploy, with clear failure handling.
- **Per-environment pipelines and locks**, not one shared lock that blocks everything.
- Keep migrations small and transactional where the database supports it.

Example:

```
flyway -url=$STAGING_DB info            # state: Success / Failed / Pending
flyway -url=$STAGING_DB repair          # clean failed entry after fixing the script
liquibase --url=$STAGING_DB release-locks
alembic current ; alembic downgrade -1

# Pre-migration snapshot (RDS)
aws rds create-db-snapshot --db-instance-identifier staging-db \
  --db-snapshot-identifier pre-mig-$(date +%Y%m%d%H%M)

-- Idempotent, expand-style migration
ALTER TABLE orders ADD COLUMN IF NOT EXISTS customer_ref VARCHAR(64);
```

What to say to the interviewer: *I stop the pipeline and lock promotion, then check the migration history table to see exactly what was applied and whether it is marked failed or locked. If it rolled back cleanly I repair and rerun; if it is partially applied I either fix forward with an idempotent script or run the down-migration or restore the pre-migration snapshot, then release locks so other environments are unblocked. To prevent it I use expand-contract migrations, automatic snapshots, dry-runs against a prod-like copy, and a separate migration job per environment instead of one shared lock.*

Q76. During a blue-green deployment, shifting traffic to green caused a spike in errors. What do you check and fix?

In simple words: You moved the audience to the new stage and people started complaining. Move them back to the old stage right away (it is still ready), then check what the new stage was missing - lights not warmed up, wrong script, not enough seats.

Detailed answer

Immediately: **shift traffic back to blue** (listener / target group / Service selector / slot swap). Blue is still running, so this is fast. Confirm errors drop. Then investigate green without users on it:

1. **Readiness vs real warm-up:** health check passed but the app was cold - empty caches, JIT not warmed, connection pools not filled, lazy loading on first request.
2. **Health check path vs real path:** `/health` returns 200 but does not test the DB, cache or downstream APIs that real requests need.
3. **Config/secret/env diff:** compare blue vs green env vars, ConfigMaps, secrets, feature flags and endpoints (e.g. green pointing to a wrong DB or missing a secret).
4. **DB schema compatibility:** green expects a migration that was not applied, or a migration broke blue/green coexistence.
5. **Sessions:** in-memory sessions or sticky cookies bound to blue are lost -> users logged out or 401/500 errors. Use a shared session store (Redis).
6. **Capacity:** green scaled smaller than blue (fewer replicas, smaller instances, HPA min too low) -> overload at 100% traffic.
7. **Downstream limits:** third-party rate limits, IP allowlists that only include blue's egress IPs, security groups, connection storms to the DB (max connections).
8. **Logs, metrics and traces by version:** which endpoints and error codes (5xx vs 4xx) appear only on green.

Prevention:

- Do a **canary step first** (1-10% to green) before 100%.
- **Automated analysis and rollback:** Argo Rollouts blue-green with pre/post-promotion analysis, CodeDeploy with CloudWatch alarms.
- **Pre-warm** green (synthetic traffic, cache warm-up) and match its capacity to blue.
- Deep health checks, config parity checks, externalized sessions.

Example:

```
# ALB: send traffic back to blue
aws elbv2 modify-listener --listener-arn $LISTENER \
  --default-actions Type=forward,TargetGroupArn=$TG_BLUE

# K8s: flip the Service back
kubectl patch svc orders -p '{"spec":{"selector":{"app":"orders","version":"blue"}}}'

# Compare config and capacity
diff <(kubectl get deploy orders-blue -o jsonpath='{.spec.template.spec.containers[0].env}') \
     <(kubectl get deploy orders-green -o jsonpath='{.spec.template.spec.containers[0].env}')
kubectl get hpa orders-blue orders-green
```

> ***What to say to the interviewer:*** *First I shift traffic back to blue, which is still running, and confirm errors drop. Then I check green offline: whether it was really warmed up or only passed a shallow health check, config and secret differences from blue, database schema compatibility, lost sessions or sticky cookies, whether green had the same capacity, and downstream rate limits or IP allowlists. To prevent it I add a canary step before full cutover, automated analysis with Argo Rollouts or CodeDeploy alarms, pre-warming and deep health checks.*

Q77. How do you know your last deploy was actually successful? Monitoring? Business metrics? Or silence?

> **In simple words:** A green pipeline only means the parcel left the warehouse. Success means the customer received it and is happy. And "nobody complained" is not proof - maybe the phone line is broken.

Detailed answer

Pipeline green is not success. Layers of evidence, from technical to business:

1. **Rollout completed:** `kubectl rollout status`, all new pods Ready, no restarts; Helm/Argo CD status Healthy and Synced.
2. **Smoke tests and synthetic checks** right after deploy: hit key endpoints and user journeys (login, checkout) - part of the pipeline, fail = rollback.
3. **Golden signals before vs after** (and canary vs baseline): error rate, p95/p99 latency, traffic, saturation (CPU, memory, DB connections). Automate with **Argo Rollouts AnalysisTemplate**, **CodeDeploy alarms** or **Kayenta**-style canary analysis.
4. **Business KPIs:** checkouts per minute, successful logins, payments processed, orders created, meter reads processed - a deploy can be technically healthy but break a business flow.
5. **Log error-rate diff:** new error messages or exception types appearing after the deploy (Loki/ELK query by version).
6. **Deploy annotations on Grafana dashboards** so every graph shows exactly when the deploy happened.

7. **SLO / error-budget burn alerts:** a fast burn right after a deploy = likely bad deploy.
8. **Track DORA metrics:** change failure rate and time to restore tell you how often "successful" deploys really were not.

Silence is not evidence: verify the alerting path itself works - alert rules loaded, Alertmanager/contact points healthy, notification templates valid, a test alert or heartbeat (dead man's switch) arriving in Teams/Slack. A broken notification channel looks exactly like "everything is fine".

Example:

```
kubectl -n prod rollout status deploy/orders-api --timeout=5m
curl -fsS https://api.example.com/health/deep && ./smoke-tests.sh prod

# Error rate new version vs before (PromQL)
sum(rate(http_requests_total{app="orders",code=~"5.."}[5m]))
  / sum(rate(http_requests_total{app="orders"}[5m]))

# Argo Rollouts analysis (fragment)
kind: AnalysisTemplate
spec:
  metrics:
  - name: error-rate
    successCondition: result[0] < 0.01
    provider:
      prometheus:
        query: sum(rate(http_requests_total{app="orders",code=~"5.."}[5m])) /
               sum(rate(http_requests_total{app="orders"}[5m]))

# Heartbeat alert: always firing, proves the alert pipeline is alive
- alert: Watchdog
  expr: vector(1)
```

> ***What to say to the interviewer:*** *A green pipeline only proves the deploy ran, so I verify in layers: rollout status and readiness, post-deploy smoke and synthetic tests, golden signals compared before and after or canary versus baseline with automated analysis, business KPIs like logins or orders, log error diffs, deploy annotations on dashboards and error-budget burn alerts, and we track change failure rate. I never treat silence as success - I make sure the alerting path itself works with heartbeat alerts and tested notification channels.*

Q78. You notice a sudden spike in failed deployments. How would you investigate?

> **In simple words:** If suddenly many parcels are failing delivery, you do not check each parcel one by one. You sort them into piles - by city, by truck, by day - and find the one thing they have in common: a closed road, a broken truck, or a new wrong address label.

Detailed answer

1. **Quantify and categorize** the failures by **stage** (build, push, deploy, health check), **service**, **environment**, **agent/runner** and **time**. A spike across many services at the same stage means a shared cause; one service means that service's own change.
2. **Find the common change** around when it started:

- a **pipeline template / shared library** version bump;
- a new **base image** or dependency version (broken upstream release);

- an **agent/runner image update** (tool versions changed, e.g. new Helm or kubectl);
- an **expired credential**, certificate or service connection (OIDC trust, token, registry password);
- an outage of a **dependency registry** (npm, Maven Central, Docker Hub rate limits, ECR);
- cloud **quota** or API rate limits;
- **cluster capacity** (pods Pending, no nodes, IP exhaustion) or a new admission policy (OPA/Kyverno rule rejecting manifests).

3. **Read the actual errors** from a few failures in each category - often they are the same message.
4. **Check trends:** DORA **change failure rate** and deployment frequency over recent weeks - a sudden jump points to a platform change; a slow rise points to quality or test gaps.
5. **Fix the common cause** (pin or roll back the template or agent image, rotate the credential, add a registry mirror, add capacity), then rerun the failed deploys.
6. **Add guardrails:** version-pinned templates and agent images rolled out gradually (canary pipelines), alerts on credential expiry, a pull-through cache for registries, capacity alerts, and a dashboard of deploy failures by stage so the next spike is spotted in minutes.

Example:

```
# GitHub Actions: failed runs since yesterday, grouped by workflow
gh run list --status failure --created ">=2026-09-23" --limit 200 \
  --json workflowName,createdAt | jq 'group_by(.workflowName)
  | map({wf: .[0].workflowName, fails: length})'

# Kubernetes side: are deploys failing on capacity or policy?
kubectl get events -A --field-selector type=Warning --sort-by=.lastTimestamp | tail -30
kubectl get pods -A --field-selector status.phase=Pending

# Change failure rate = failed deployments / total deployments (per week)
```

> ***What to say to the interviewer:*** *I categorize the failures by stage, service, environment, agent and time to see whether it is one service or a shared cause, then look for the common change - a pipeline template or agent image update, a new base image, an expired credential or service connection, a registry outage, quota, or cluster capacity and admission policies. I confirm with the actual error messages and the change failure rate trend, fix the shared cause and rerun, then add guardrails like pinned templates and agent images, credential-expiry alerts, registry caching and a deploy-failure dashboard.*

Q79. Jenkins is failing to push a Docker image to the registry. How do you troubleshoot?

> **In simple words:** A delivery van can't drop a parcel at the warehouse. Is the van's permit expired (auth), is it at the wrong address (tag/repo name), is the road blocked (network), or is the warehouse refusing that parcel (permissions, immutable tags, size)?

Detailed answer

1. **Read the exact error in the console log.** It usually points straight at the layer:

- `unauthorized` / `denied` / `no basic auth credentials` -> authentication. ECR login tokens expire after **12 hours**, so run `aws ecr get-login-password` in the pipeline every time; the Jenkins credential or IAM role may be wrong or expired.

- `denied: ... not authorized to perform ecr:InitiateLayerUpload` -> the IAM policy is missing push actions (`ecr:PutImage`, `InitiateLayerUpload`, `UploadLayerPart`, `CompleteLayerUpload`, `BatchCheckLayerAvailability`).

- `name unknown` / `repository does not exist` -> ECR needs the repo created first (Terraform it), or the tag doesn't include the full registry prefix.

- `tag invalid: ... already exists` -> **immutable tags** are enabled, so use unique tags (the Git SHA or build number).

- `i/o timeout` / `TLS handshake timeout` -> network: proxy, DNS, security group egress, or missing VPC endpoints (`ecr.api`, `ecr.dkr`, S3) for private agents.

2. **Agent side:** is the Docker daemon running, and does the `jenkins` user have socket permissions (`permission denied ... docker.sock`)? Is there disk space for layers (`docker system df`)?

3. **Image size and timeouts:** huge layers on slow links, so optimise the image and retry.

4. **Reproduce manually** on the same agent as the same user, to separate a pipeline problem from an environment problem.

Example:

```
stage('Push') {
  steps {
    withAWS(role: 'jenkins-ecr-push', region: 'ap-south-1') {
      sh '''
        REG=123456789012.dkr.ecr.ap-south-1.amazonaws.com
        aws ecr get-login-password | docker login --username AWS --password-stdin $REG
        docker tag meter-api:${GIT_COMMIT} $REG/meter-api:${GIT_COMMIT}
        docker push $REG/meter-api:${GIT_COMMIT}
      '''
    }
  }
}
```

```
# manual checks on the agent, as the jenkins user
sudo -u jenkins docker info >/dev/null && echo "daemon ok"
aws sts get-caller-identity
aws ecr describe-repositories --repository-names meter-api
nc -zv 123456789012.dkr.ecr.ap-south-1.amazonaws.com 443
```

> ***What to say to the interviewer:*** *I read the exact push error first because it tells me the layer. Unauthorized means an expired ECR token or wrong credentials, so I log in freshly each run with the IAM role. AccessDenied on upload actions means the IAM policy. "Repository does not exist" or a tag conflict means the repo isn't created or tags are immutable. Timeouts mean proxy, DNS or missing VPC endpoints. I also check that the Docker daemon, socket permissions and disk on the agent are fine, and reproduce the push manually as the jenkins user.*